Basic reinforcement learning

08:35 video · English narration · Guide-linked examples

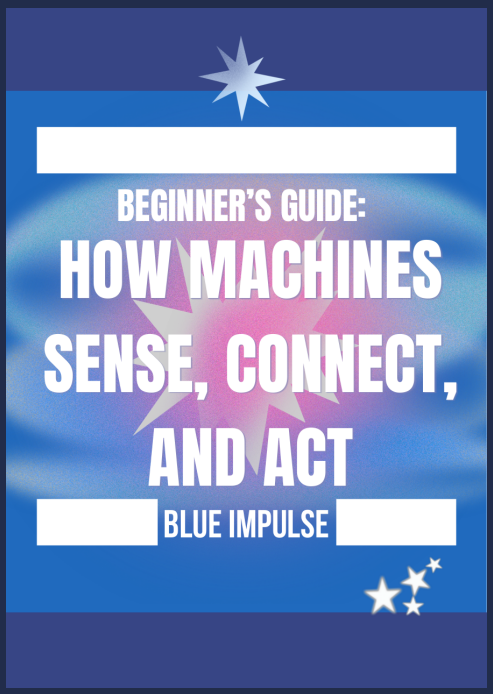
Learn with the book

1. Name the learning loop

2. Balance exploration and exploitation

3. Calculate one Q-value update

Keep the 96-page Blue Impulse study guide open. Its printed page numbers match the PDF viewer. The screenshots in this tutorial are exact crops from that book.



Guide pages: 24, 25, 26, 27, 28, 31.

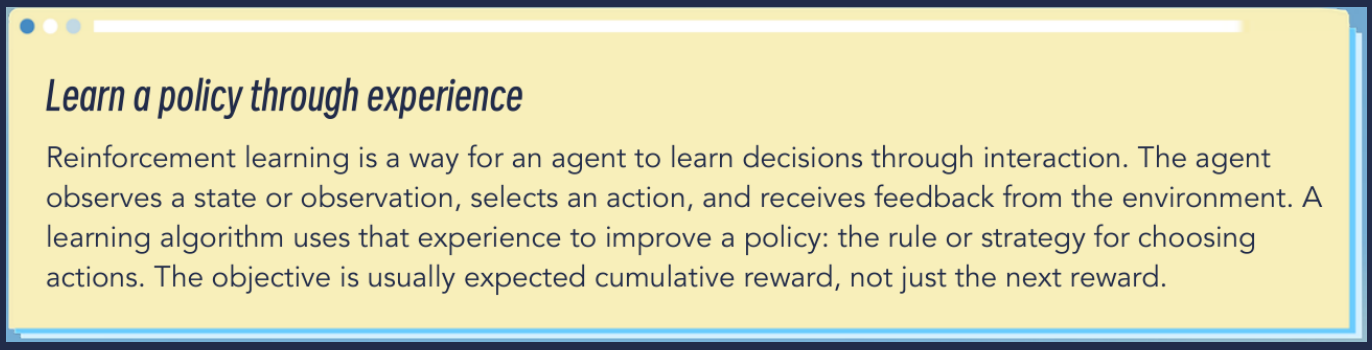
How to use this PDF

Follow the timestamp at the top of each lesson page to find the matching video section. Read the book excerpt, work through the teaching text, and stop at each question before reading the answer. The complete narration is included here, with separate editable scripts and subtitle files in the package.

Practice file: BlueImpulse_Demonstrations.py
Requires Python 3.9+ and no extra packages. The source guide, scripts, subtitle sidecars, and editable video project are included in the tutorial package.

How does a rover learn a route?

Video 00:00–00:34 / Study guide page 25



Exact excerpt from the Blue Impulse study guide

Have you ever wondered how a rover could learn a useful route without someone writing every turn?

Imagine a simulated rover choosing between two paths, trying moves, and receiving points for the outcomes.

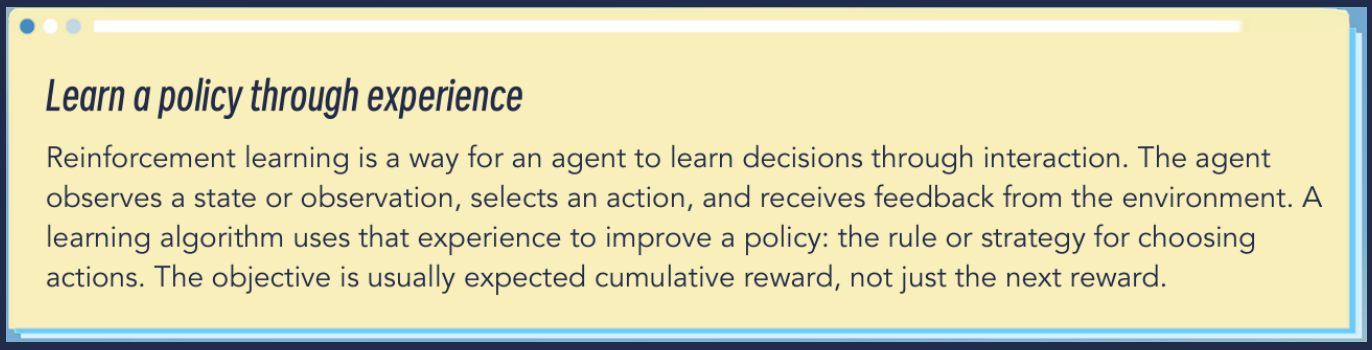
The important question is how that experience changes its next decision.

Keep your Blue Impulse study guide open at page twenty-five. We will work directly through the book.

By the end, you will calculate a learning update yourself and explain what the result actually means.

Today's route

Video 00:34–01:11 / Study guide page 25



Exact excerpt from the Blue Impulse study guide

Today you will identify an agent, an environment, a state, an action, a reward, and a policy.

You will compare exploration with exploitation, then work through the numerical example on page twenty-eight.

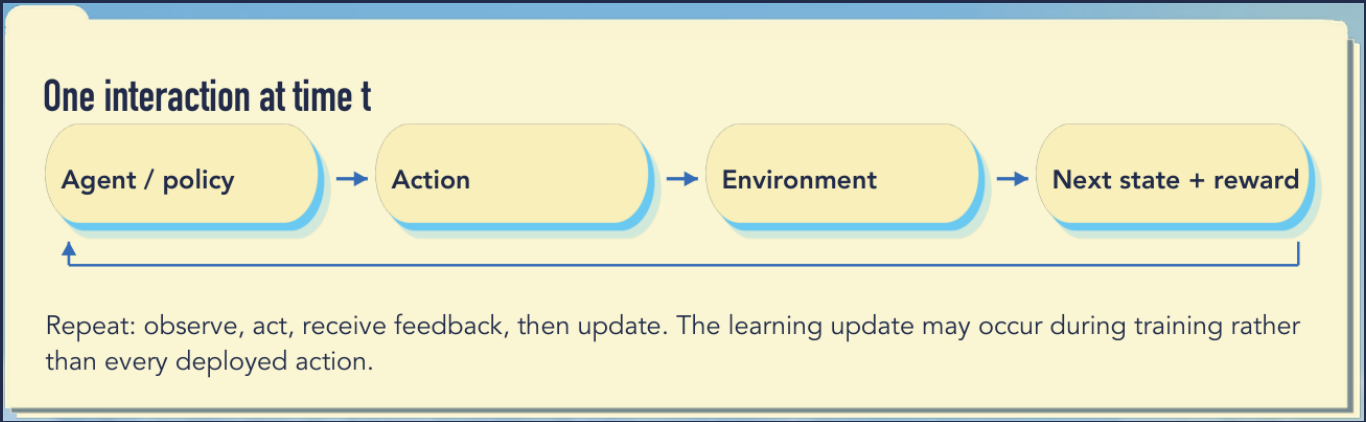
Welcome to the tutorial on basic reinforcement learning by Blue Impulse.

Have paper ready for one calculation. The tutorial PDF includes the same worked steps and practice question.

Treat our rover as an illustration. We are explaining a learning method, not claiming every real rover learns this way.

Follow the learning loop

Video 01:11–01:52 / Study guide page 25



Exact excerpt from the Blue Impulse study guide

Look at the diagram on page twenty-five, titled One interaction at time t.

The agent selects an action. The environment changes and returns the next state and a reward.

The learning algorithm uses that experience to improve the policy, which is the strategy for choosing actions.

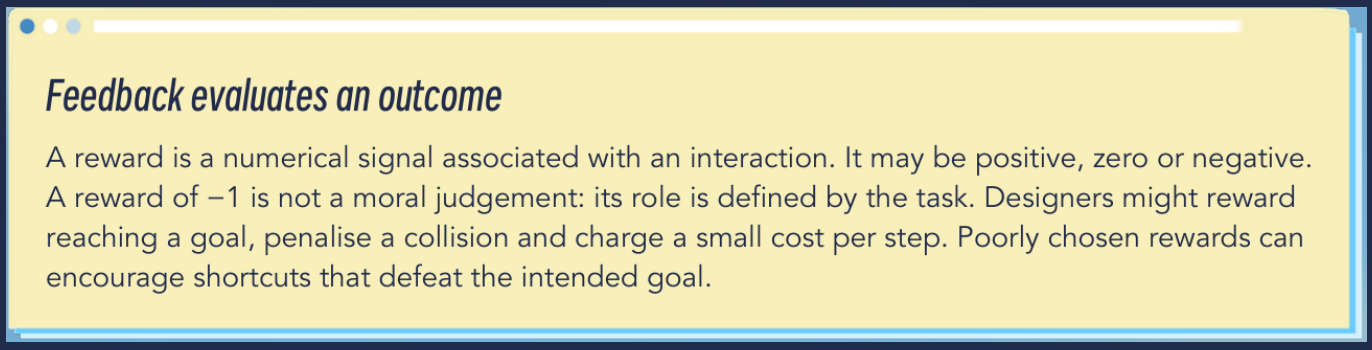
For our grid, the state could be the current square, and an action could be moving right.

The reward is a number associated with the result. It is feedback, not a complete instruction manual.

Notice the return arrow. Learning depends on connecting an action to what happened after it.

Reward, policy, value

Video 01:52–02:34 / Study guide page 26



Exact excerpt from the Blue Impulse study guide

Page twenty-six separates three ideas that are easy to confuse: reward, policy, and value.

Think of a journey. The reward scores an outcome now, while the policy chooses the next move.

A value estimate predicts how much return a choice may lead to, including later outcomes.

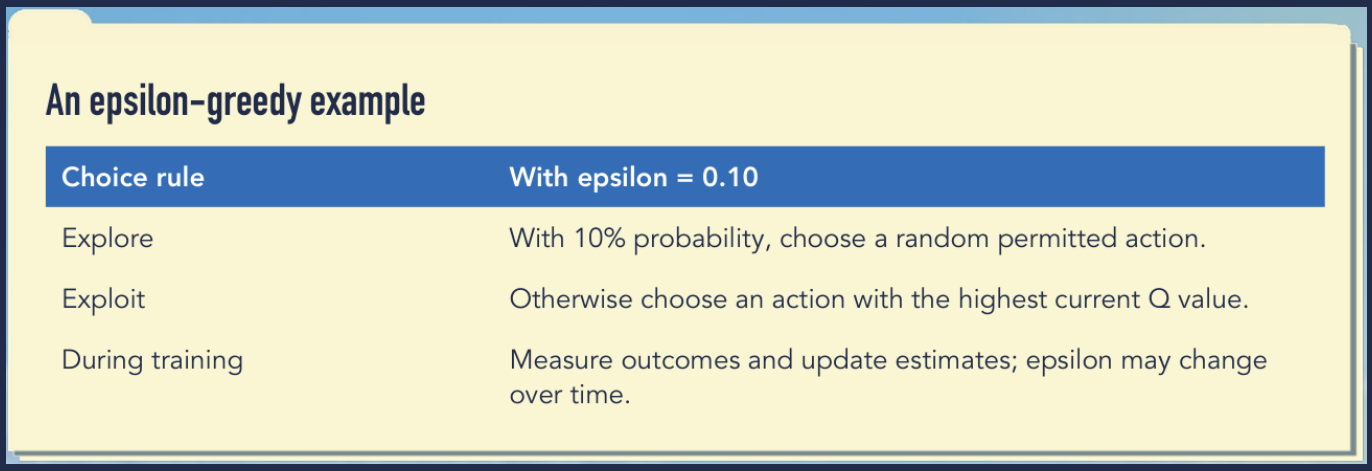
A route with a small cost now can still be worthwhile if it reaches a better destination.

That is why choosing the biggest immediate reward is not always the best long-term decision.

The book also warns that poorly chosen rewards can encourage shortcuts that miss the designer's real goal.

The restaurant analogy

Video 02:34–03:24 / Study guide page 27



An epsilon-greedy example

Choice rule	With epsilon = 0.10
Explore	With 10% probability, choose a random permitted action.
Exploit	Otherwise choose an action with the highest current Q value.
During training	Measure outcomes and update estimates; epsilon may change over time.

Exact excerpt from the Blue Impulse study guide

Turn to page twenty-seven and use the book's restaurant analogy.

Ordering the dish you currently believe is best is exploitation. Trying a different dish is exploration.

Exploration gathers information because your current estimates might be wrong.

In the book's epsilon-greedy example, epsilon is zero point one, giving a ten percent exploration probability.

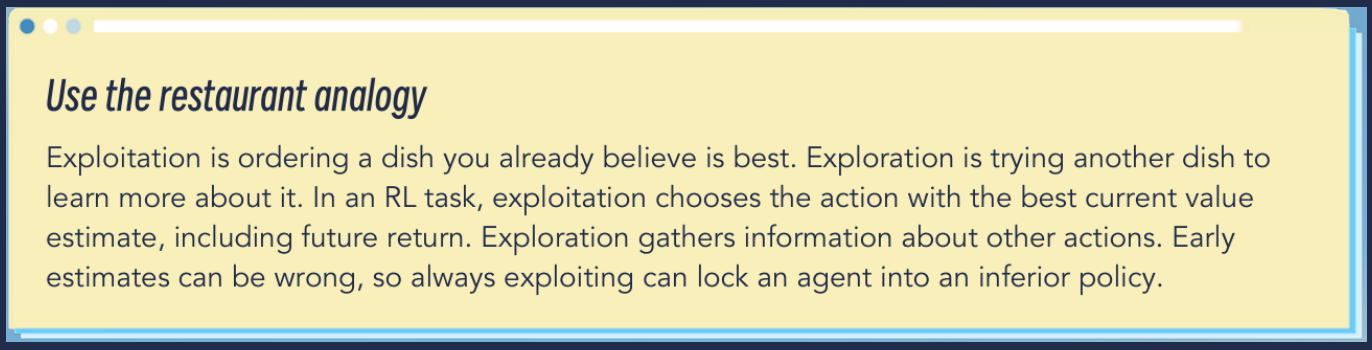
Otherwise, the agent chooses an action with the highest current Q value.

These percentages describe probabilities over choices, not a promise that exactly one out of every ten choices explores.

In a real physical system, exploration also needs safety constraints. A simulation is a useful place to experiment.

Is the familiar route certainly best?

Video 03:24–04:05 / Study guide page 27



Exact excerpt from the Blue Impulse study guide

Pause and think. One route has worked twice. Another route has never been tried.

Does that prove the first route is better? Take a few seconds before we reveal the answer.

No. The evidence is incomplete, so the familiar route is not certainly the better one.

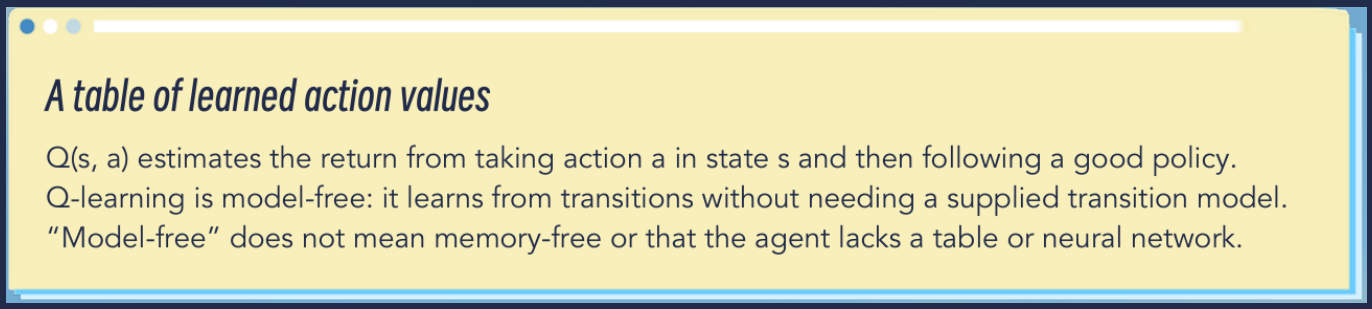
Exploration may improve the estimate, but it has a cost. The design must balance information and performance.

Write one sentence explaining why exploitation uses estimated future return, rather than only the next reward.

Pause first: Two successes versus an untried route: proof?
Check: No. The evidence is incomplete.

What is a Q value?

Video 04:05–04:49 / Study guide page 28



Exact excerpt from the Blue Impulse study guide

Now open page twenty-eight, Update a Q value. Q associates a state and an action with an estimated return.

The book calls Q-learning model-free because it does not require a supplied model of environment transitions.

Model-free does not mean memory-free. The estimates still live in a table or another representation.

The update combines an old estimate with a target derived from new experience.

Alpha controls how far the estimate moves toward that target. Gamma discounts the future-value contribution.

We will use exactly the numbers in the guide, so you can check each line against the page.

Work through the book's update

Video 04:49–05:45 / Study guide page 28

Work through one illustrative update

Quantity	Value
Old Q / alpha	2 / 0.5
Reward / gamma / best next Q	1 / 0.9 / 4
Target	$1 + 0.9 \times 4 = 4.6$
New Q	$2 + 0.5 \times (4.6 - 2) = 3.3$

Exact excerpt from the Blue Impulse study guide

Start with an old Q value of two and a learning rate, alpha, of zero point five.

The reward is one, gamma is zero point nine, and the best next Q value is four.

First calculate the target: one plus zero point nine times four. The target is four point six.

Next subtract the old estimate. Four point six minus two gives an error of two point six.

Multiply that error by alpha. Half of two point six is one point three.

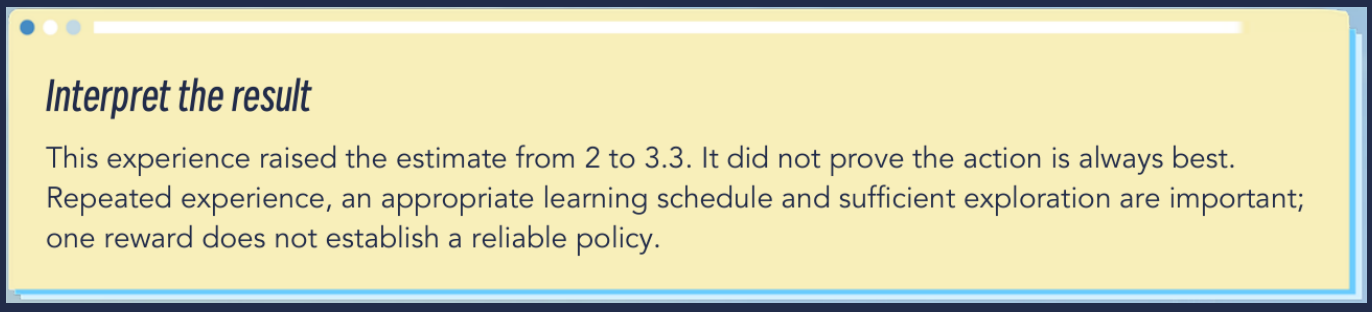
Finally add the change to the old value. Two plus one point three gives three point three.

Watch the estimate move halfway toward the target. That halfway movement is what alpha equals one half means here.

The supporting Python file repeats this calculation and prints every intermediate value for you to inspect.

A better estimate, not a proof

Video 05:45–06:34 / Study guide page 28



Exact excerpt from the Blue Impulse study guide

Return to the final paragraph on page twenty-eight. This experience raised the estimate from two to three point three.

It did not prove that the action is always best. One observation is not a reliable policy.

Repeated experience, an appropriate learning schedule, and enough exploration are still important.

There is also a special case: when the next state is terminal, the future-value term is zero.

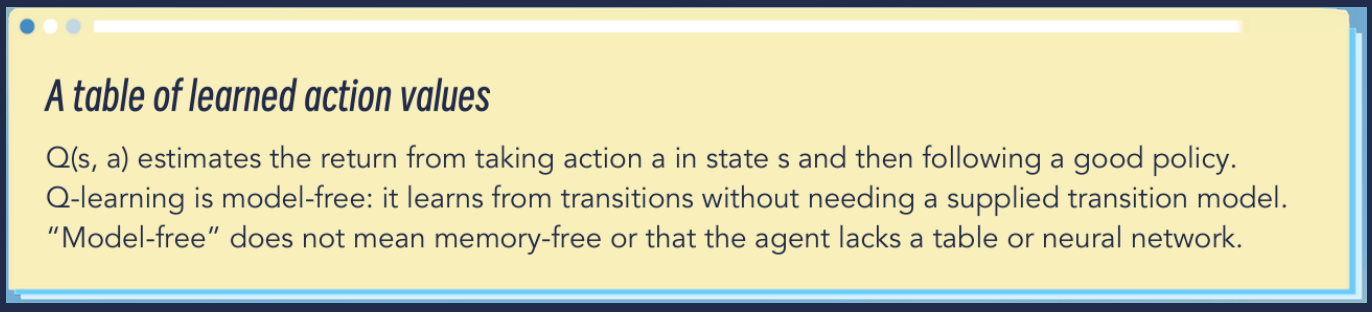
That prevents the calculation from inventing future reward after the episode has ended.

Compare these two ideas: updating an estimate from evidence, and guaranteeing the outcome of the next action.

They are different claims, and only the first follows from our calculation.

Try a terminal state

Video 06:34–07:17 / Study guide page 28



Exact excerpt from the Blue Impulse study guide

Your turn. Keep the old value at two, alpha at one half, and the reward at one.

Make the next state terminal. What is the new Q value? Pause the video if you need more time.

The future contribution is zero, so the target is one. The error is one minus two, or negative one.

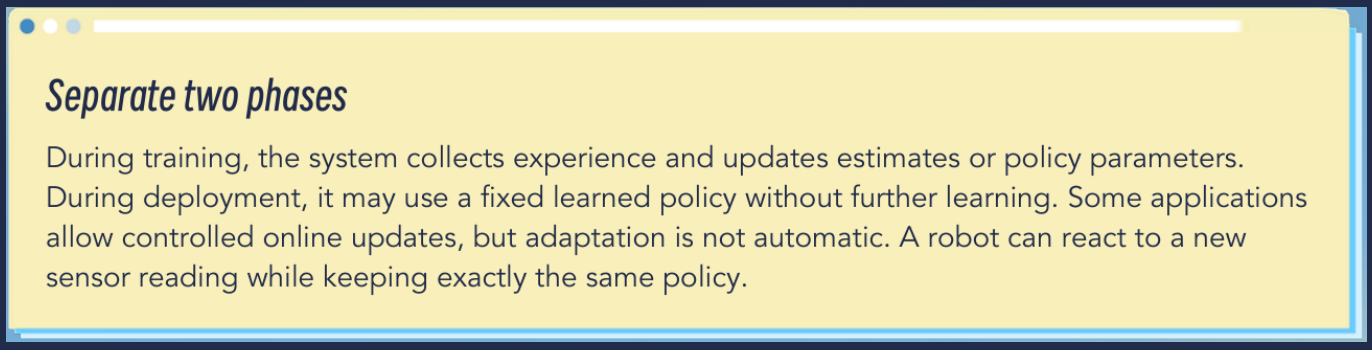
Half that error is negative one half. Add it to two, giving a new value of one point five.

The estimate falls because the observed target is lower. A learning update can decrease a value as well as increase it.

Pause first: Terminal state: old Q = 2, alpha = 0.5, reward = 1
Check: New Q = 2 + 0.5 × (1 - 2) = 1.5

Experience changes estimates

Video 07:17–08:07 / Study guide page 31



Exact excerpt from the Blue Impulse study guide

Let's recap using pages twenty-five through twenty-eight. The agent acts, receives feedback, and learns a policy from experience.

Reward scores an outcome. Value estimates return. Exploration improves information, while exploitation uses the best current estimates.

Our worked Q update produced three point three, and our terminal-state practice produced one point five.

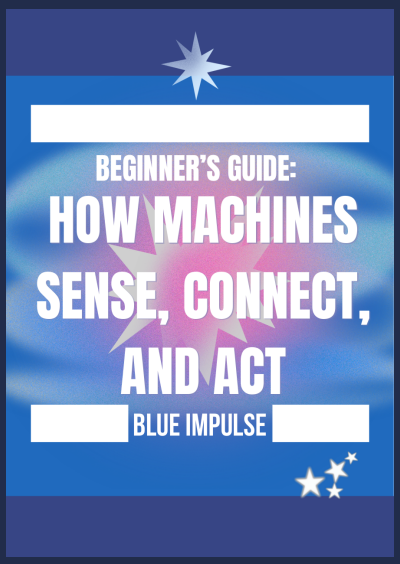
Page thirty-one adds an important distinction: running an already learned policy is not the same as training it online.

A fixed sensor-response rule can control a machine without learning at all.

Before moving on, explain the loop to someone else and run the included calculation with one changed reward.

Blue Impulse study guide

Video 08:07–08:35 / Study guide page 1



This tutorial was taught from the Blue Impulse study guide, especially pages twenty-four to twenty-eight and thirty-one.

The cover on screen is the original cover of your book.

Use the companion tutorial PDF, narration script, subtitles, and Python demonstration to practise at your own pace.

Blue Impulse study guide. Learn the reasoning, then test it with an example.

How hardware enables AI

09:04 video · English narration · Guide-linked examples

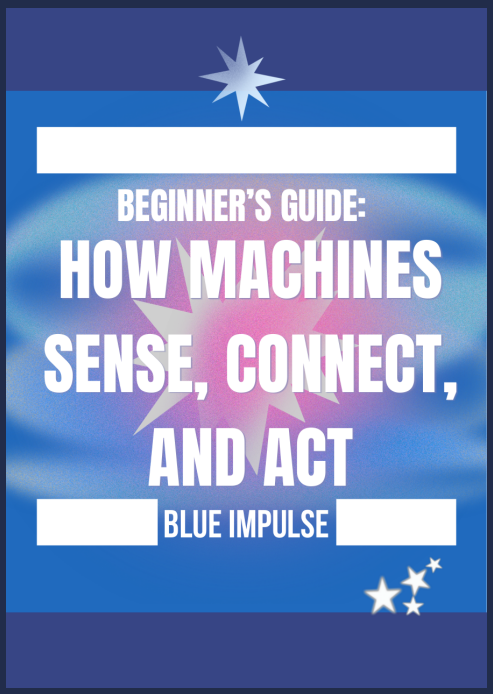
Learn with the book

1. Trace sensing to action

2. Explain memory and processing roles

3. Diagnose a hardware bottleneck

Keep the 96-page Blue Impulse study guide open. Its printed page numbers match the PDF viewer. The screenshots in this tutorial are exact crops from that book.



Guide pages: 35, 36, 39, 44, 46, 86.

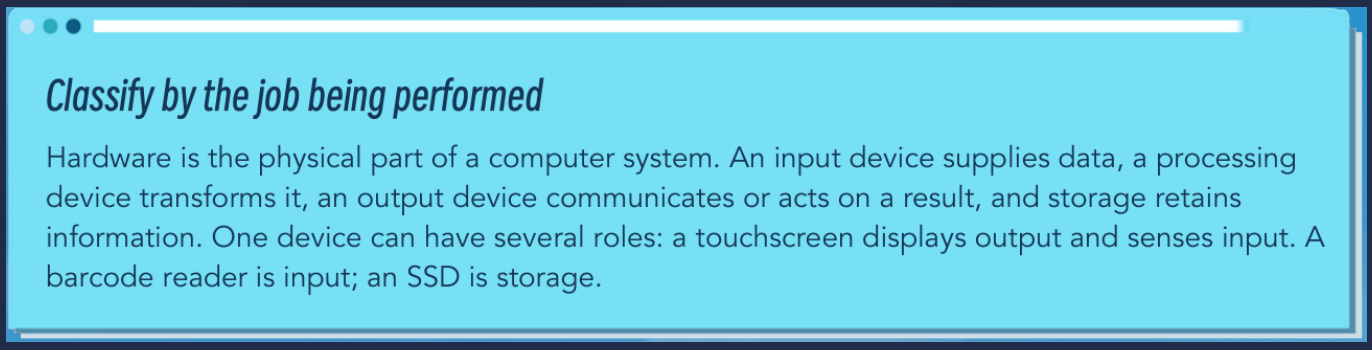
How to use this PDF

Follow the timestamp at the top of each lesson page to find the matching video section. Read the book excerpt, work through the teaching text, and stop at each question before reading the answer. The complete narration is included here, with separate editable scripts and subtitle files in the package.

Practice file: BlueImpulse_Demonstrations.py
Requires Python 3.9+ and no extra packages. The source guide, scripts, subtitle sidecars, and editable video project are included in the tutorial package.

What gives an AI decision a body?

Video 00:00–00:36 / Study guide page 35



Exact excerpt from the Blue Impulse study guide

Have you ever wondered how a prediction inside a computer becomes a robot moving across a room?

A model cannot see, remember, calculate, or turn a wheel without physical components doing those jobs.

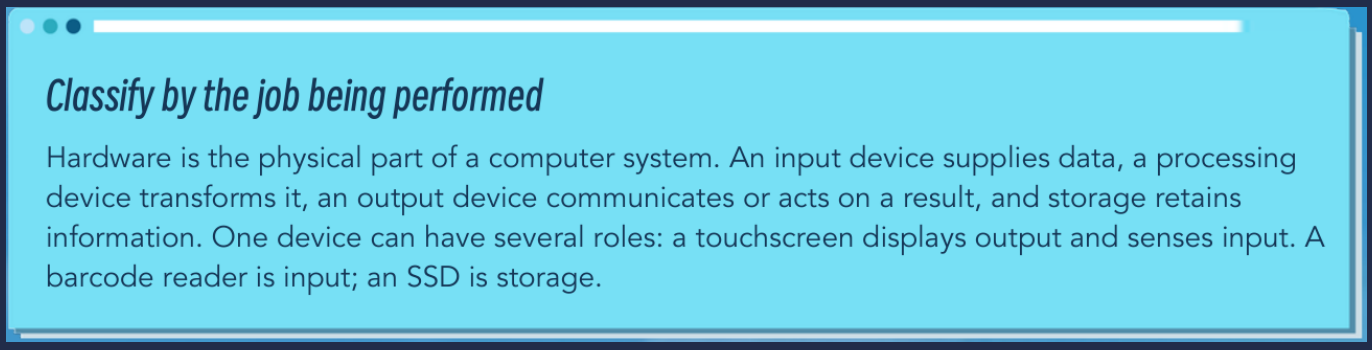
Today we will follow one observation through the hardware, all the way to an action and new feedback.

Open the Blue Impulse study guide at page thirty-five, Hardware gives software a body.

Keep page forty-four nearby too. It explains why a control signal and motor power are different things.

Today's route

Video 00:36–01:11 / Study guide page 35



Exact excerpt from the Blue Impulse study guide

You will trace the sensing, memory, processing, and actuation path of a small robot.

You will explain why memory movement can limit a fast processor and what a pipeline actually improves.

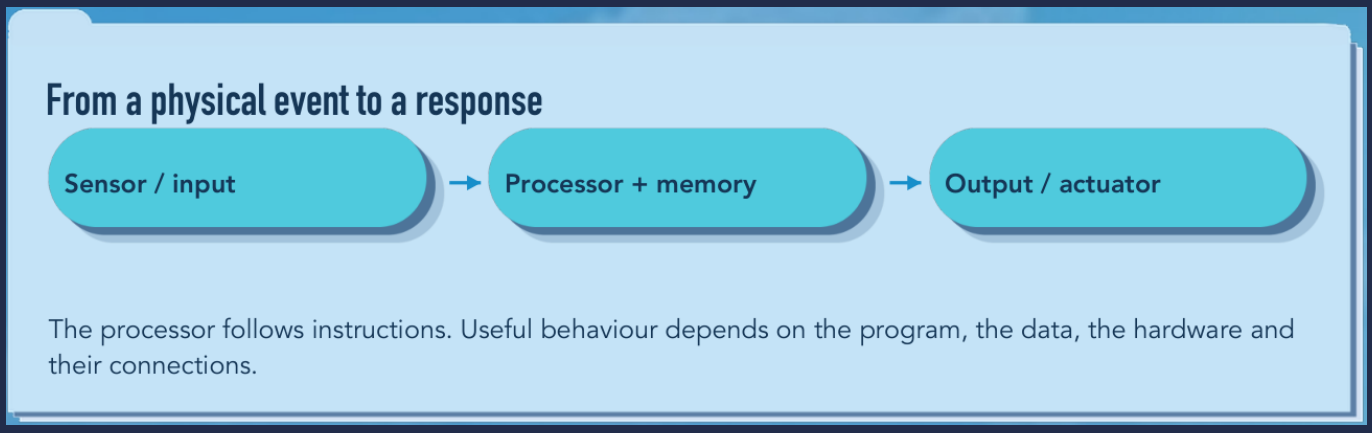
You will also choose hardware by its job, rather than assuming every AI system needs the same chip.

Welcome to the tutorial on how hardware enables AI by Blue Impulse.

We are using the guide as our lesson: read the pictured excerpts, then follow the demonstrations beside them.

Follow the physical path

Video 01:11–01:56 / Study guide page 35



Exact excerpt from the Blue Impulse study guide

Page thirty-five groups hardware by the job it performs: input, processing, output, and storage.

For our robot, the sensor supplies observations. Memory holds inputs, model parameters, and working results.

The processor evaluates the program or model, and an output communicates the resulting decision.

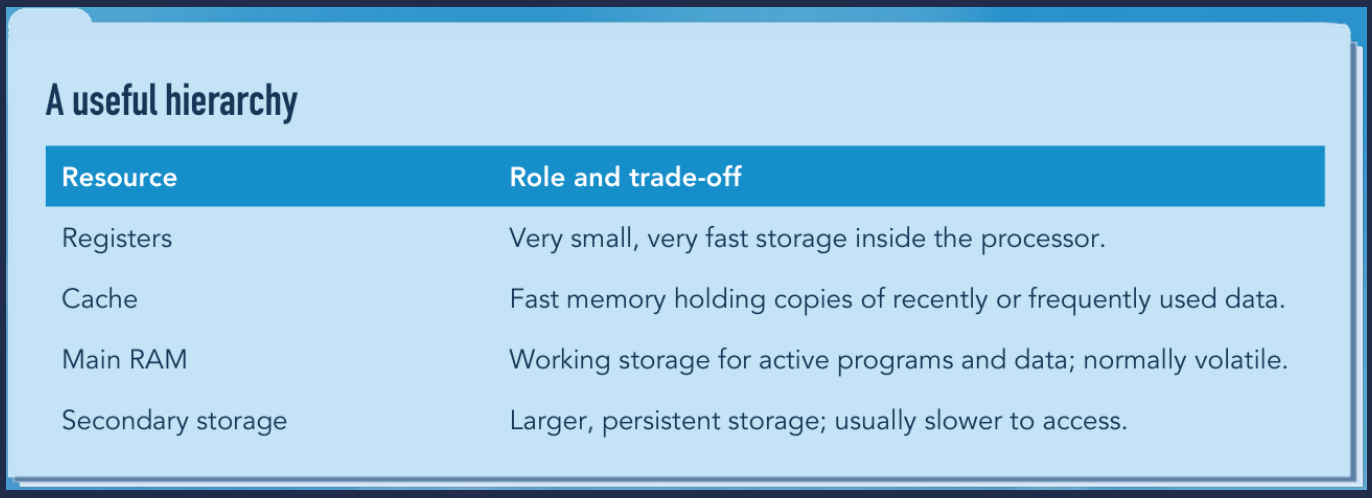
If the robot needs to move, an actuator must turn that decision into physical motion.

Notice that one device can have multiple roles. A touchscreen both displays output and senses input.

The diagram is a chain of responsibilities, not a claim that every box must be a separate physical chip.

Use the kitchen analogy

Video 01:56–02:38 / Study guide page 36



A useful hierarchy

Resource	Role and trade-off
Registers	Very small, very fast storage inside the processor.
Cache	Fast memory holding copies of recently or frequently used data.
Main RAM	Working storage for active programs and data; normally volatile.
Secondary storage	Larger, persistent storage; usually slower to access.

Exact excerpt from the Blue Impulse study guide

Use a kitchen to remember the memory hierarchy on page thirty-six.

Registers are like the tiny workspace immediately in your hands. Cache keeps useful ingredients close to the cook.

Main memory is the larger active workbench, while persistent storage is more like the pantry.

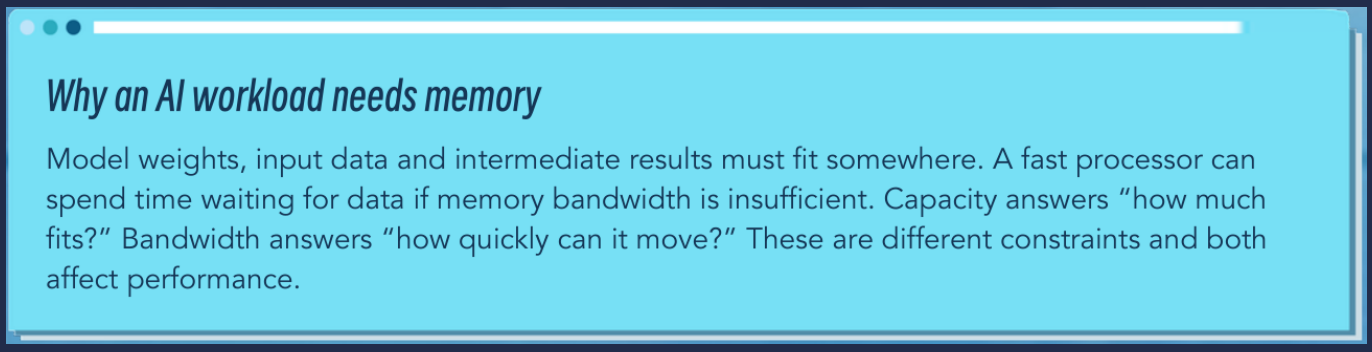
The analogy is about access and capacity, not a literal map of computer components.

A huge pantry does not help if ingredients arrive too slowly. Similarly, a processor can wait for data.

Capacity asks how much fits. Memory bandwidth asks how quickly data can move. These are different questions.

The processor may be waiting

Video 02:38–03:28 / Study guide page 36



Exact excerpt from the Blue Impulse study guide

Look at the final paragraph on page thirty-six. Model weights, inputs, and intermediate results must fit somewhere.

When memory bandwidth is insufficient, a fast processor may spend time waiting instead of doing useful arithmetic.

Adding storage capacity alone does not automatically remove a memory-transfer bottleneck.

The table also distinguishes cache, main RAM, and secondary storage. They are not interchangeable names for one resource.

Both static RAM and dynamic RAM are volatile, although their circuits and refresh requirements differ.

For this tutorial, focus on the system question: is the limiting factor how much fits, or how quickly it arrives?

Match the processor to the work

Video 03:28–04:16 / Study guide page 46

Different processing strengths

Hardware	Useful role
CPU	General control, branching and diverse program tasks.
GPU	High-throughput parallel arithmetic for suitable workloads.
NPU / TPU-type accelerator	Specialised tensor operations with hardware-specific efficiency.
RAM and storage	Hold weights, inputs, intermediate results and saved models.

Exact excerpt from the Blue Impulse study guide

Now compare the processing roles in the table on page forty-six.

A CPU handles general control, branches, and diverse tasks. A GPU can accelerate suitable parallel arithmetic workloads.

Specialised accelerators can efficiently perform supported tensor operations, but their benefits depend on the actual workload.

Simple tabular reinforcement learning can run on an ordinary CPU. AI does not always require a GPU.

Training changes model parameters. Inference uses an existing model to produce an output.

A model can be trained on a large server and deployed on a smaller device if the deployment requirements fit.

Overlap work, increase throughput

Video 04:16–05:19 / Study guide page 39

A simplified four-stage pipeline

Instruction	Cycle 1	2	3	4	5
I1	Fetch	Decode	Execute	Write	—
I2	—	Fetch	Decode	Execute	Write
I3	—	—	Fetch	Decode	Execute

Exact excerpt from the Blue Impulse study guide

Page thirty-nine uses laundry to explain pipelining. One load can dry while a later load is washing.

In our simplified processor, the stages are fetch, decode, execute, and write.

Instruction one enters fetch in cycle one. In cycle two, it decodes while instruction two enters fetch.

Keep stepping forward. The instructions occupy different stages, so several instructions make progress during the same cycle.

After the pipeline fills, this ideal example completes one instruction per cycle, assuming no stalls.

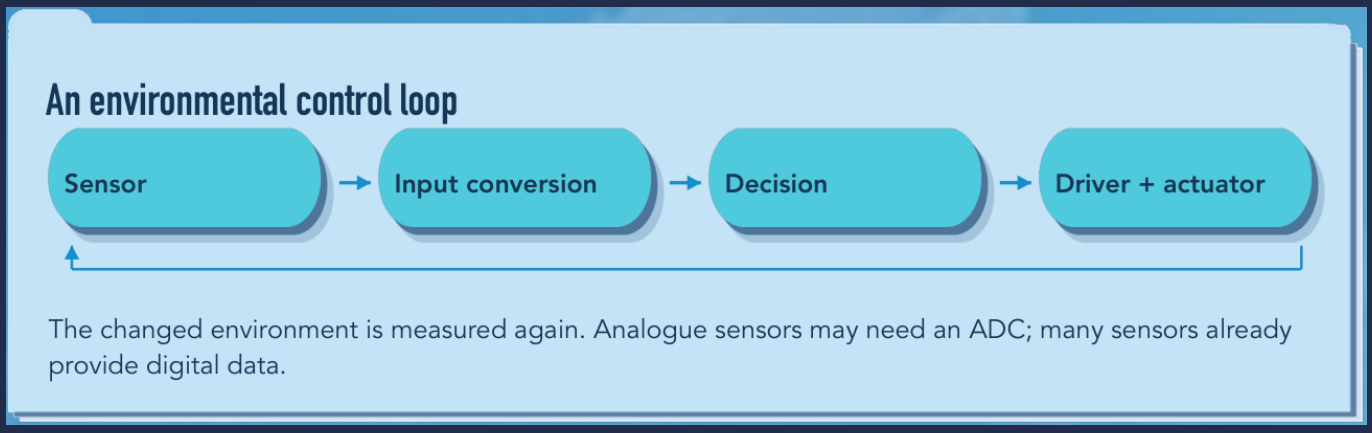
It does not make an individual four-stage instruction take just one cycle from start to finish.

Real pipelines can pause for dependencies, uncertain branches, or shared resources. The guide names these as hazards.

Use the included Python table to compare completion cycles rather than relying only on the moving picture.

A signal is not motor power

Video 05:19–06:06 / Study guide page 44



Exact excerpt from the Blue Impulse study guide

Turn to page forty-four and trace the environmental control loop.

An analogue sensor may need an analogue-to-digital converter. Many sensors already provide digital measurements.

The processor evaluates a decision, then a driver or power stage supplies what the actuator needs.

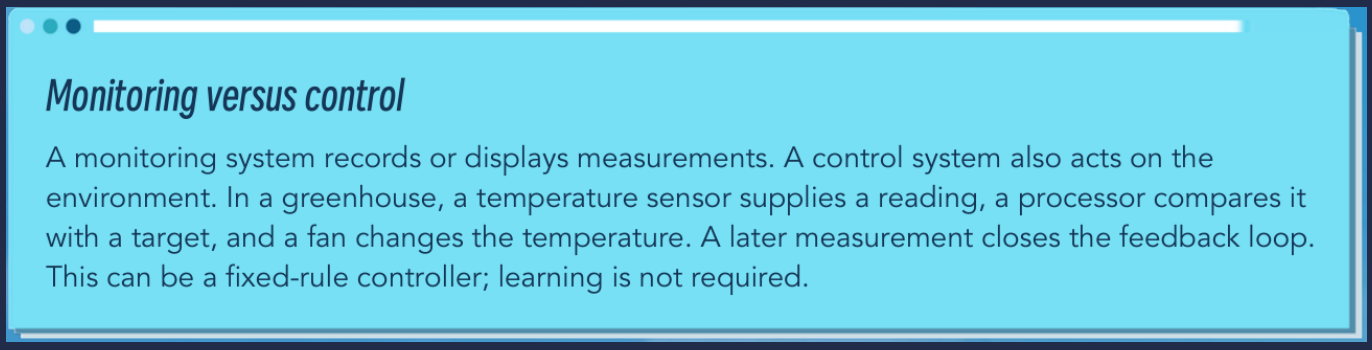
Some actuators use a digital interface or pulse-width modulation. They do not all require a digital-to-analogue converter.

Pulse-width modulation is timed switching, not the same thing as converting a value into an analogue voltage.

A control signal can tell a motor what to do without being able to supply the current that moves it.

Close the loop

Video 06:06–06:50 / Study guide page 44



Exact excerpt from the Blue Impulse study guide

Imagine the robot detects an obstacle, selects a turn, and sends a command to its motor driver.

The driver powers the motor, the wheels move, and the sensor measures the changed situation.

That final measurement closes the feedback loop. The command alone does not prove the movement succeeded.

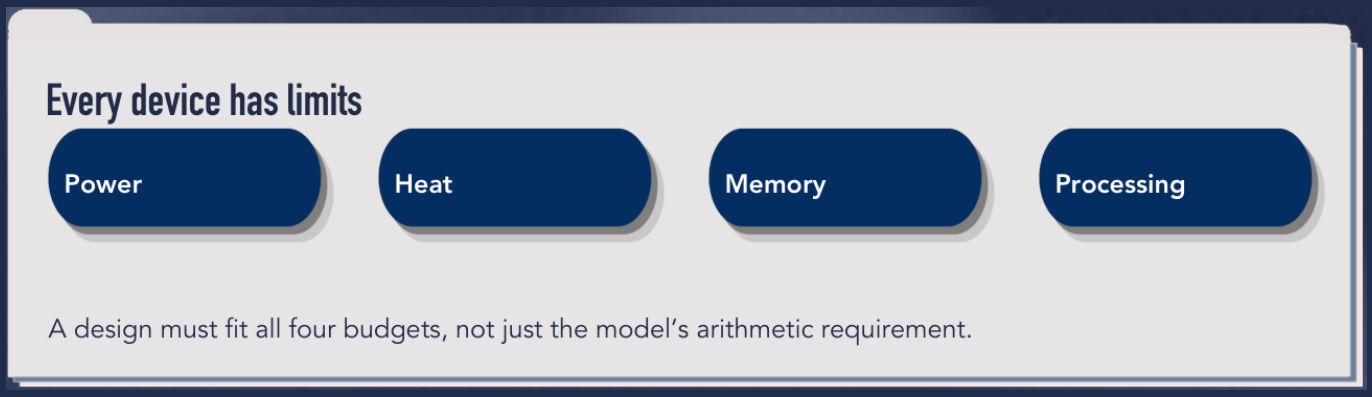
A wheel might slip, a sensor might be noisy, or the actuator might reach a mechanical limit.

The book also separates monitoring from control. Displaying a measurement monitors; changing the environment adds control.

This loop can use a fixed rule or a learned policy. The hardware path alone does not tell you which.

Will a faster GPU fix this?

Video 06:50–07:43 / Study guide page 86



Exact excerpt from the Blue Impulse study guide

Pause and diagnose a system. Its processor repeatedly waits for data, and its motor driver cannot supply enough current.

Would replacing the processor with a faster GPU solve both problems? Think before the answer appears.

No. Memory movement and actuator power are separate limits that a new GPU does not automatically remove.

Measure the path, identify the limiting stage, and match the intervention to that evidence.

Page eighty-six broadens the discussion to power, heat, memory, and physical constraints.

An advertised arithmetic rate describes one capability. It is not a complete measurement of the robot's useful performance.

Pause first: Waiting for data + insufficient motor current
Check: Fix the memory path and power stage.

Hardware enables every step

Video 07:43–08:34 / Study guide page 46

Different processing strengths

Hardware	Useful role
CPU	General control, branching and diverse program tasks.
GPU	High-throughput parallel arithmetic for suitable workloads.
NPU / TPU-type accelerator	Specialised tensor operations with hardware-specific efficiency.
RAM and storage	Hold weights, inputs, intermediate results and saved models.

Exact excerpt from the Blue Impulse study guide

Recap the path from page thirty-five: observations enter, memory holds information, processing evaluates it, and outputs communicate or act.

Page thirty-six explains why memory capacity and bandwidth both matter. Page thirty-nine separates pipeline throughput from instruction latency.

Page forty-four explains conversion, control signals, and actuator power. Page forty-six compares processing strengths.

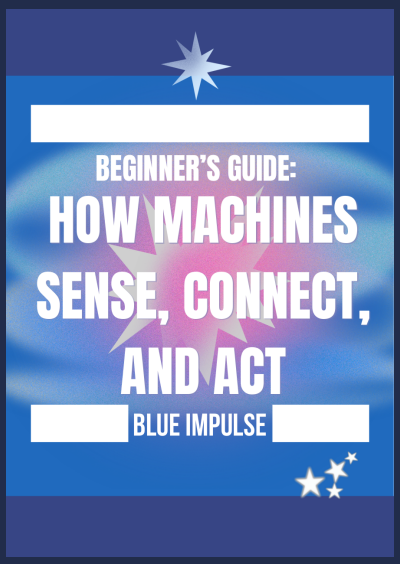
Choose hardware by the workload, deadlines, power budget, and physical task.

To check your understanding, draw a robot's path from camera to wheel, and label one possible bottleneck at each stage.

Then explain why a more powerful chip does not, by itself, create a learning algorithm.

Blue Impulse study guide

Video 08:35–09:04 / Study guide page 1



This tutorial used the Blue Impulse study guide, pages thirty-five, thirty-six, thirty-nine, forty-four, forty-six, and eighty-six.

The original book cover is shown on screen as our credit.

The companion tutorial PDF and pipeline demonstration help you repeat the lesson without replaying every explanation.

Blue Impulse study guide. Trace the whole system before choosing the hardware.

Network challenges

09:14 video · English narration · Guide-linked examples

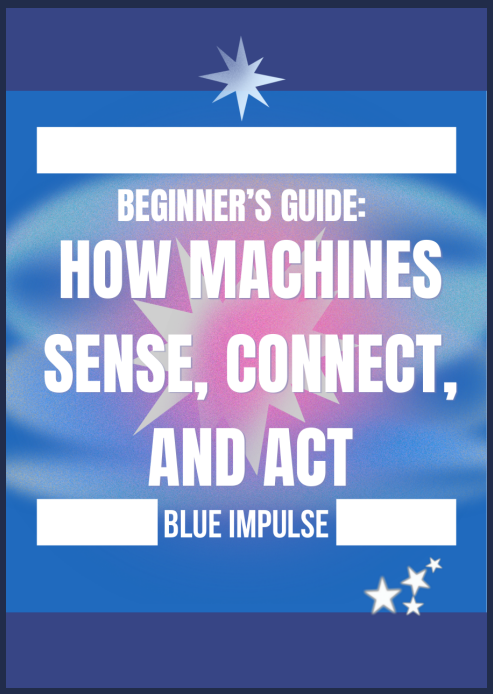
Learn with the book

1. Distinguish five network measures

2. Calculate the effect of delay

3. Choose a response to the actual problem

Keep the 96-page Blue Impulse study guide open. Its printed page numbers match the PDF viewer. The screenshots in this tutorial are exact crops from that book.



Guide pages: 56, 59, 61, 62, 63, 74, 88.

How to use this PDF

Follow the timestamp at the top of each lesson page to find the matching video section. Read the book excerpt, work through the teaching text, and stop at each question before reading the answer. The complete narration is included here, with separate editable scripts and subtitle files in the package.

Practice file: BlueImpulse_Demonstrations.py
Requires Python 3.9+ and no extra packages. The source guide, scripts, subtitle sidecars, and editable video project are included in the tutorial package.

Why can fast Wi-Fi still feel slow?

Video 00:00–00:39 / Study guide page 62

Four problems to distinguish

Metric	What it tells you
Bandwidth / capacity	How much data a link can carry per second.
Throughput	The successful transfer rate actually achieved.
Latency	How long a transfer or response takes.
Jitter / loss	Variation in delay / missing packets.

Exact excerpt from the Blue Impulse study guide

Have you ever wondered why a connection advertised as fast can still make a video call freeze or a controller respond late?

The advertised rate describes only part of the journey. Delay, variation, loss, and shared demand also matter.

Open your Blue Impulse study guide at page sixty-two, Capacity is not responsiveness.

We will use that page to diagnose problems instead of treating every network problem as a need for more bandwidth.

Then we will calculate the effect of delay using the book's example on page seventy-four.

Today's route

Video 00:39–01:14 / Study guide page 62

Four problems to distinguish

Metric	What it tells you
Bandwidth / capacity	How much data a link can carry per second.
Throughput	The successful transfer rate actually achieved.
Latency	How long a transfer or response takes.
Jitter / loss	Variation in delay / missing packets.

Exact excerpt from the Blue Impulse study guide

Today you will distinguish bandwidth, throughput, latency, jitter, and packet loss.

You will use a numerical example to connect delay with a physical consequence.

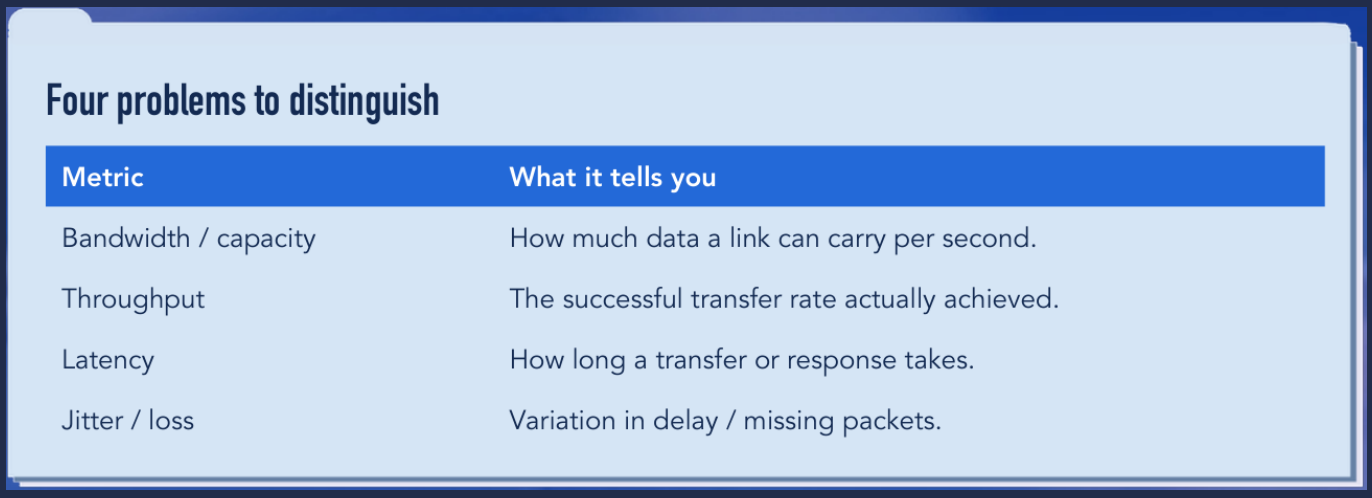
You will choose responses such as compression, fewer round trips, buffering, or local control for specific problems.

Welcome to the tutorial on network challenges by Blue Impulse.

Keep a calculator or paper ready. Every book example has a page citation, and the tutorial PDF includes the practice answers.

Name the problem before fixing it

Video 01:14–01:56 / Study guide page 62



Four problems to distinguish

Metric	What it tells you
Bandwidth / capacity	How much data a link can carry per second.
Throughput	The successful transfer rate actually achieved.
Latency	How long a transfer or response takes.
Jitter / loss	Variation in delay / missing packets.

Exact excerpt from the Blue Impulse study guide

Read the table at the top of page sixty-two. Bandwidth describes how much data a link can carry per second.

Throughput is the successful transfer rate actually achieved. It can be lower than the link's capacity.

Latency is how long a transfer or response takes. Jitter is variation in that delay.

Packet loss means some packets do not arrive as expected.

These measures answer different questions. A connection can carry a lot of data and still take a long time to respond.

When describing an experiment, specify what you measured: one-way delay, round-trip delay, or complete task response time.

Use the road analogy

Video 01:56–02:42 / Study guide page 62

Four problems to distinguish

Metric	What it tells you
Bandwidth / capacity	How much data a link can carry per second.
Throughput	The successful transfer rate actually achieved.
Latency	How long a transfer or response takes.
Jitter / loss	Variation in delay / missing packets.

Exact excerpt from the Blue Impulse study guide

Use the guide's road analogy. A wider road can carry more cars at once, like greater bandwidth.

How many cars actually arrive each second is closer to throughput.

How long one journey takes is closer to latency. Uneven arrival times illustrate jitter, while missing deliveries illustrate loss.

Widening a road does not shorten the distance to the destination or remove every queue.

Likewise, increasing link capacity does not erase propagation time, server processing, or unnecessary back-and-forth exchanges.

The analogy helps separate the ideas, but measure the actual network before deciding which factor dominates.

Shared demand changes the result

Video 02:42–03:27 / Study guide page 56

Collision detection and avoidance

CSMA/CD	CSMA/CA
Historically used on shared, half-duplex Ethernet.	Used by Wi-Fi to manage shared radio access.
Listen, transmit, detect collision, back off and retry.	Listen, wait/back off, transmit and use acknowledgements.
Modern switched full-duplex Ethernet has no collisions on the link.	RTS/CTS is an optional mechanism, not a compulsory step every time.

Exact excerpt from the Blue Impulse study guide

Now connect page sixty-two with the media and contention discussion on page fifty-six.

Wireless devices may share a medium, compete for transmission time, and experience interference.

Heavy demand can create queues. Retransmissions after loss can use more capacity and add delay.

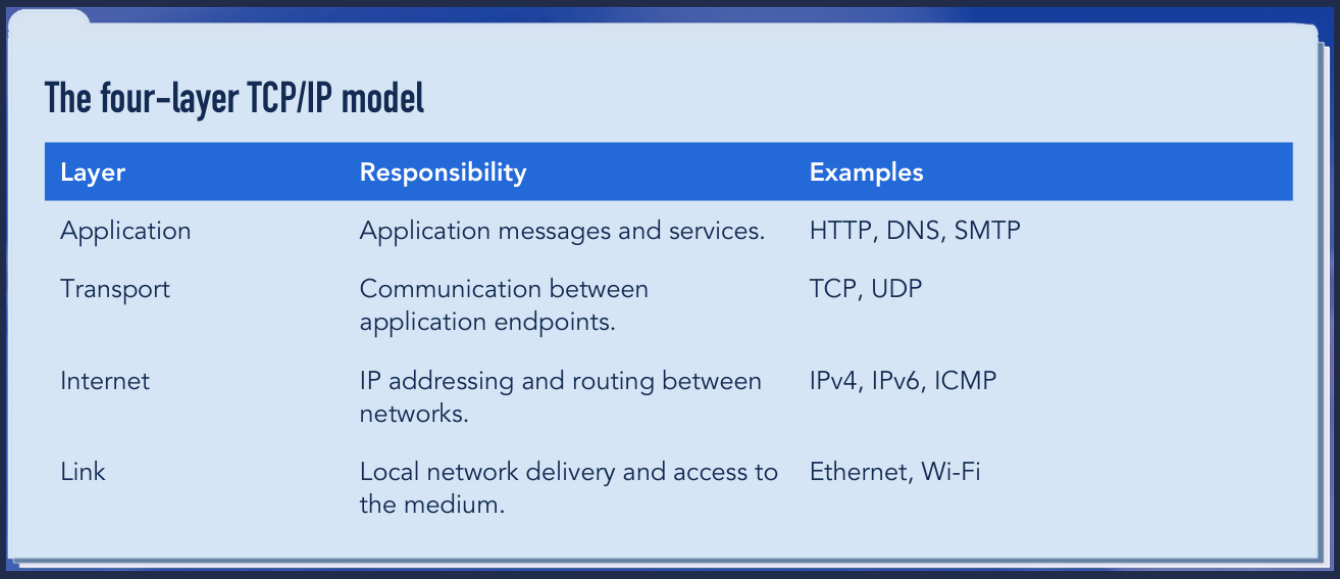
Modern switched full-duplex Ethernet should not be described as suffering the old shared-hub collision model.

The useful diagnosis depends on the network technology, the traffic, and the end-to-end path.

Try testing near the access point, with fewer competing transfers, and at different times before blaming one advertised number.

Reliability has a timing cost

Video 03:27–04:20 / Study guide page 59



The four-layer TCP/IP model

Layer	Responsibility	Examples
Application	Application messages and services.	HTTP, DNS, SMTP
Transport	Communication between application endpoints.	TCP, UDP
Internet	IP addressing and routing between networks.	IPv4, IPv6, ICMP
Link	Local network delivery and access to the medium.	Ethernet, Wi-Fi

Exact excerpt from the Blue Impulse study guide

Page fifty-nine explains TCP and IP. IP routes packets, while TCP can provide an ordered, reliable byte stream.

Reliability can require acknowledgements and retransmission, so recovery may add delay.

A transport choice does not by itself guarantee an application deadline or protect every message from an attacker.

Some applications prefer timely delivery and handle missing data themselves. Others need complete, correctly ordered transfer.

Read the application's requirements before selecting a protocol or interpreting a performance measurement.

Pages sixty-three through sixty-five discuss security separately. Confidentiality, integrity, and availability are distinct from simply having a fast link.

What happens during the delay?

Video 04:20–05:24 / Study guide page 74

Illustrative delay distance at 30 m/s

Delay before response	Distance travelled
500 ms = 0.500 s	30 × 0.500 = 15 metres
50 ms = 0.050 s	30 × 0.050 = 1.5 metres
5 ms = 0.005 s	30 × 0.005 = 0.15 metres

Exact excerpt from the Blue Impulse study guide

Turn to page seventy-four. The guide uses a constant speed of thirty metres per second for an illustrative calculation.

Distance travelled during delay equals speed multiplied by delay. First convert milliseconds into seconds.

Five hundred milliseconds is zero point five seconds. Thirty times zero point five gives fifteen metres.

Fifty milliseconds is zero point zero five seconds. That gives one point five metres.

Five milliseconds is zero point zero zero five seconds. That gives zero point one five metres.

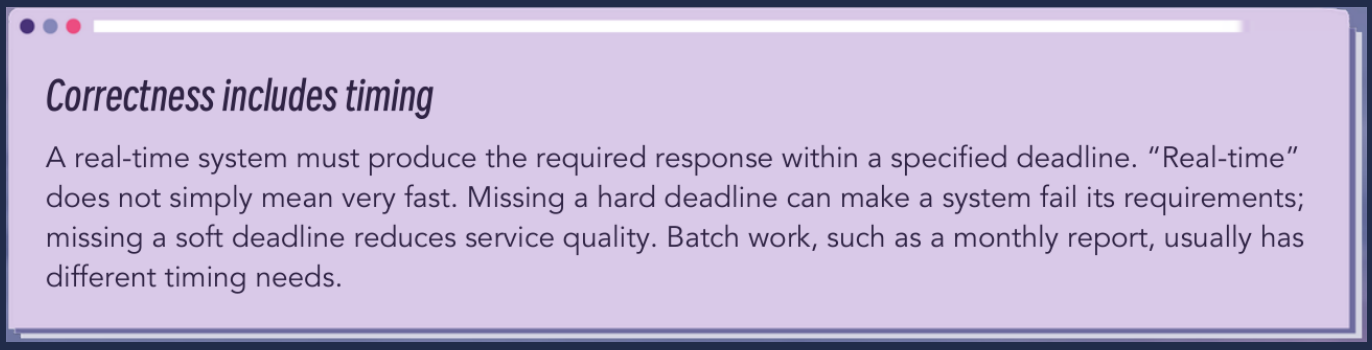
Watch the three distances grow from the same starting point. We changed delay while keeping speed fixed.

These are distances before a response begins, not complete stopping distances and not safety guarantees.

Sensing, processing, actuation, and physical braking can contribute additional delay or distance in a real system.

Real-time means a deadline

Video 05:24–06:13 / Study guide page 74



Exact excerpt from the Blue Impulse study guide

Read the first paragraph on page seventy-four. A real-time system must respond within a specified deadline.

Real-time does not simply mean very fast. Correctness includes whether the result arrives in time for its task.

Missing a hard deadline can violate the system's requirements. Missing a soft deadline reduces quality of service.

Moving a computation locally may remove some communication delay, but does not automatically guarantee a safe outcome.

Use this distinction when evaluating a remote controller: what is its deadline, and what happens if the response is late?

A good average is insufficient when the important problem is an occasional deadline miss.

Match the remedy to the symptom

Video 06:13–07:04 / Study guide page 62

Four problems to distinguish

Metric	What it tells you
Bandwidth / capacity	How much data a link can carry per second.
Throughput	The successful transfer rate actually achieved.
Latency	How long a transfer or response takes.
Jitter / loss	Variation in delay / missing packets.

Exact excerpt from the Blue Impulse study guide

Return to the final paragraph on page sixty-two and match each response to a problem.

When capacity is limited, compress or summarise suitable data. When round trips dominate, reduce unnecessary exchanges.

Timestamps help detect stale measurements. A playback buffer can smooth jitter, but the buffer adds delay.

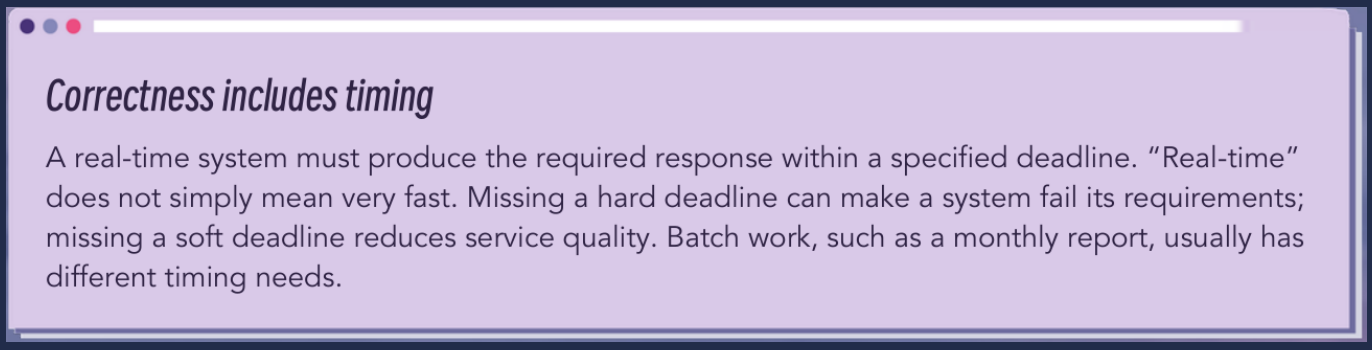
Reconnect logic and redundancy can improve resilience, while designed local control reduces dependence on an unavailable remote link.

Page eighty-eight reminds us that growing a system adds load and coordination challenges, not just more devices.

Every remedy has a trade-off. Diagnose, change one factor, and measure again using the metric that matters.

Calculate a new delay

Video 07:04–07:56 / Study guide page 74



Exact excerpt from the Blue Impulse study guide

Your turn. At thirty metres per second, how far does an object travel during a two hundred millisecond delay?

Convert the units before multiplying. Take a few seconds, or pause the video to work it out.

Two hundred milliseconds is zero point two seconds. Thirty times zero point two is six metres.

This is only distance during the delay. It does not include the rest of a real stopping process.

Now consider a separate question: would adding a playback buffer always help a time-critical controller? No, because buffering adds delay.

The right network design depends on the task, not a single universally best setting.

Pause first: 30 m/s × 200 ms = ?
Check: 30 × 0.200 = 6 metres during delay

Measure the whole task

Video 07:56–08:45 / Study guide page 62

Four problems to distinguish

Metric	What it tells you
Bandwidth / capacity	How much data a link can carry per second.
Throughput	The successful transfer rate actually achieved.
Latency	How long a transfer or response takes.
Jitter / loss	Variation in delay / missing packets.

Exact excerpt from the Blue Impulse study guide

Let's recap page sixty-two. Capacity is the limit, throughput is achieved transfer, latency is delay, jitter is variation, and loss is missing data.

Our page seventy-four demonstration gave fifteen, one point five, and zero point one five metres during three different delays.

Those numbers illustrate timing; they are not total stopping distances.

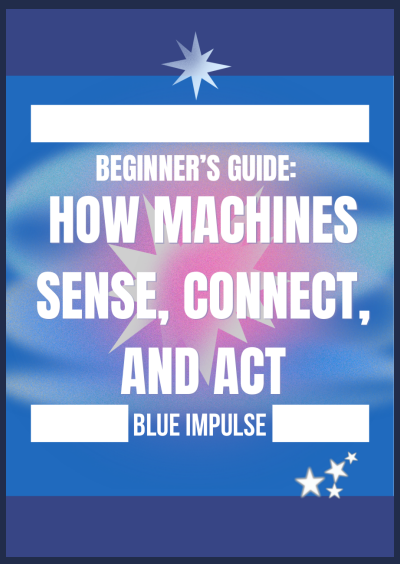
Use the road analogy to keep the measures distinct, then replace the analogy with an actual measurement.

For practice, describe a slow application, name the metric you would test, and justify one targeted improvement.

Run the supporting delay calculator to check a second speed and explain the units in your answer.

Blue Impulse study guide

Video 08:45–09:14 / Study guide page 1



This tutorial was taught from the Blue Impulse study guide, especially pages fifty-six, fifty-nine, sixty-two, seventy-four, and eighty-eight.

The original cover on screen credits the book we used throughout the lesson.

Keep the companion tutorial PDF, subtitles, narration, and delay calculator for revision.

Blue Impulse study guide. Measure the whole task, then choose the fix.

Networking for behaviorist AI

09:38 video · English narration · Guide-linked examples

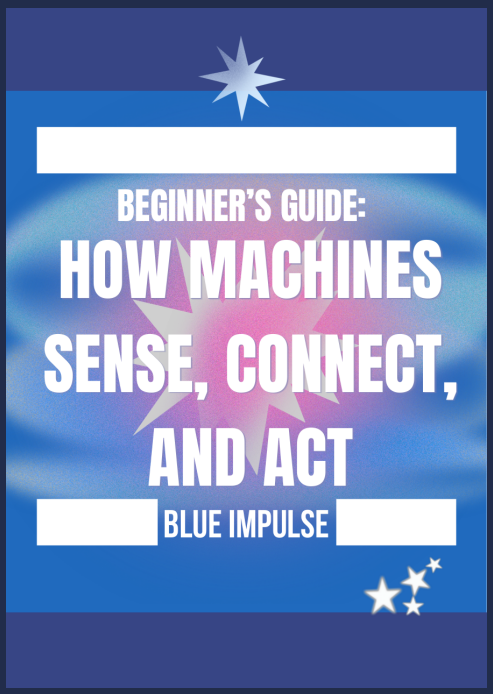
Learn with the book

1. Separate local control from shared services

2. Interpret network messages safely

3. Associate feedback with its original action

Keep the 96-page Blue Impulse study guide open. Its printed page numbers match the PDF viewer. The screenshots in this tutorial are exact crops from that book.



Guide pages: 24, 25, 72, 74, 75, 77, 78, 89.

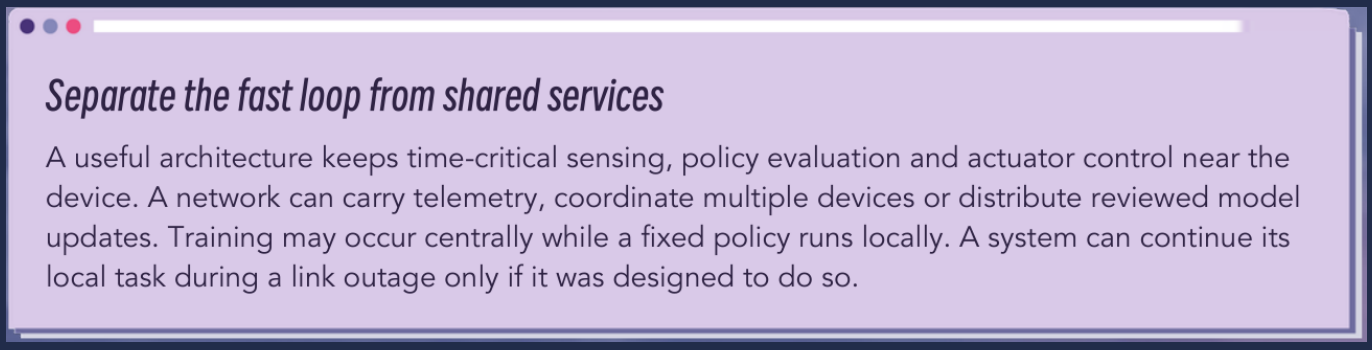
How to use this PDF

Follow the timestamp at the top of each lesson page to find the matching video section. Read the book excerpt, work through the teaching text, and stop at each question before reading the answer. The complete narration is included here, with separate editable scripts and subtitle files in the package.

Practice file: BlueImpulse_Demonstrations.py
Requires Python 3.9+ and no extra packages. The source guide, scripts, subtitle sidecars, and editable video project are included in the tutorial package.

Should a robot wait for the cloud?

Video 00:00–00:39 / Study guide page 75



Exact excerpt from the Blue Impulse study guide

Have you ever wondered what a connected robot should do when its network drops during a task?

Sending every decision to a distant server may create a delay exactly when the robot needs a timely response.

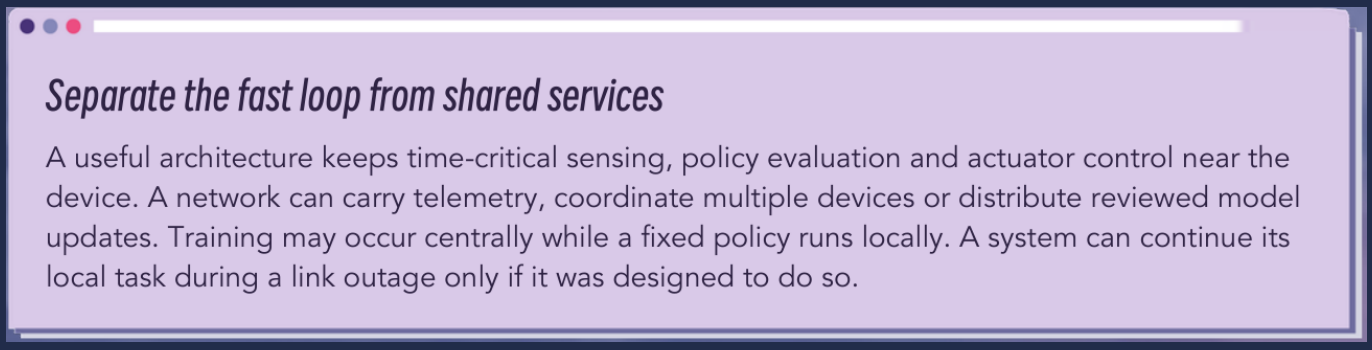
Open the Blue Impulse study guide at page seventy-five, Networking for a behaviour-based agent.

We will design the responsibilities of a small warehouse robot, then examine the messages it sends and receives.

The lesson is about making the connection useful without assuming a network is always available or a message is always current.

Today’s route

Video 00:39–01:16 / Study guide page 75



Exact excerpt from the Blue Impulse study guide

You will separate a fast local control loop from shared analysis, coordination, and training services.

You will explain why messages need context such as identity, time, sequence, units, and policy version.

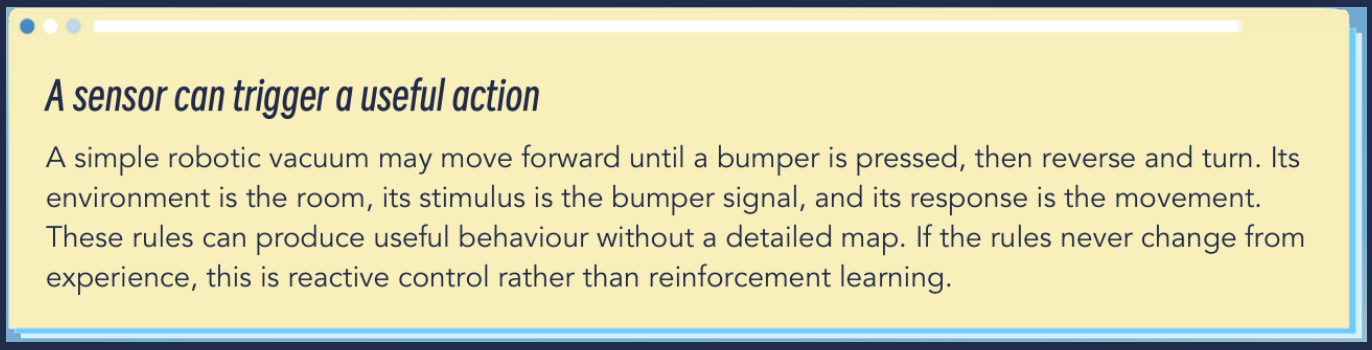
You will also connect delayed learning feedback to the action that produced it.

Welcome to the tutorial on networking for behaviorist AI by Blue Impulse.

We will use pages seventy-four, seventy-five, and eighty-nine, with an illustrative packet-checking demonstration you can run yourself.

Reactive behaviour is not always learning

Video 01:16–02:04 / Study guide page 24



A sensor can trigger a useful action

A simple robotic vacuum may move forward until a bumper is pressed, then reverse and turn. Its environment is the room, its stimulus is the bumper signal, and its response is the movement. These rules can produce useful behaviour without a detailed map. If the rules never change from experience, this is reactive control rather than reinforcement learning.

Exact excerpt from the Blue Impulse study guide

Begin with the distinction on page twenty-four. A reactive agent may map a sensed condition directly to an action.

A fixed rule such as avoiding a detected obstacle can produce useful behaviour without learning from reward.

Page twenty-five adds reinforcement learning: experience is used to improve a policy.

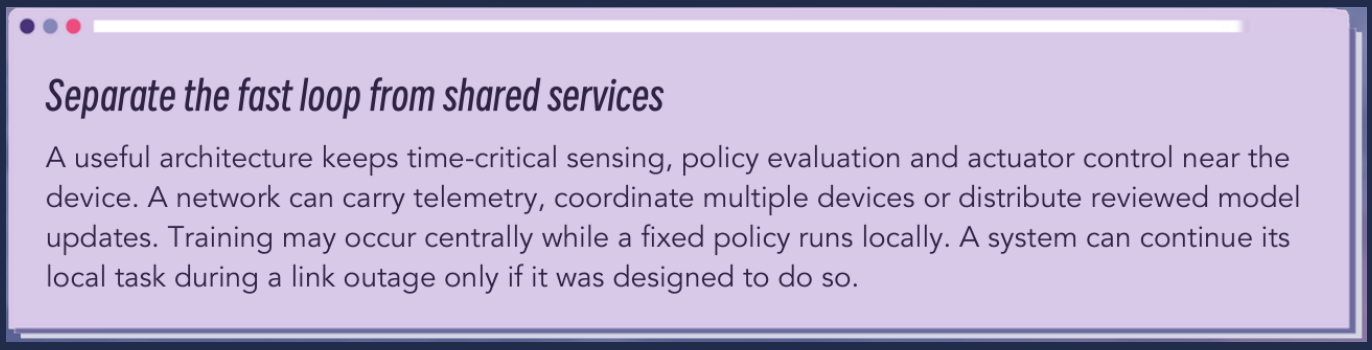
A network connection does not turn a fixed rule into a learning algorithm, and not every connected agent learns online.

For our warehouse example, think of a locally evaluated policy with separate, reviewed updates from a server.

We will label architectural choices clearly instead of treating them as properties of every AI system.

Keep the time-critical loop close

Video 02:04–02:55 / Study guide page 75



Exact excerpt from the Blue Impulse study guide

Read the first paragraph on page seventy-five. Time-critical sensing, policy evaluation, and actuator control can stay near the device.

Our robot observes an obstacle, evaluates its current policy, and sends a control signal to the actuator hardware.

This local path need not wait for every cloud exchange.

Page seventy-four explains the reason: a real-time task must meet a specified deadline, not just be fast on average.

Local processing removes some network dependency, but it still needs adequate compute, trustworthy sensing, and suitable physical controls.

Local does not automatically mean safe. The requirements and failure behaviour still have to be designed and checked.

Let the network do useful shared work

Video 02:55–03:44 / Study guide page 89

Three deployment patterns

Location	Benefit	Cost or limit
On-device edge	Low communication delay; local data.	Limited local resources and maintenance.
Nearby gateway / fog	Aggregates local devices.	Another service to size and manage.
Remote cloud	Shared compute and flexible capacity.	Network dependency, latency and transfer cost.

Exact excerpt from the Blue Impulse study guide

Now read the warehouse example on page eighty-nine. Each robot handles immediate motion control locally.

A shared service can allocate tasks, coordinate inventory, and analyse selected telemetry from several devices.

Imagine a local driver following a route while a dispatcher coordinates deliveries across the fleet.

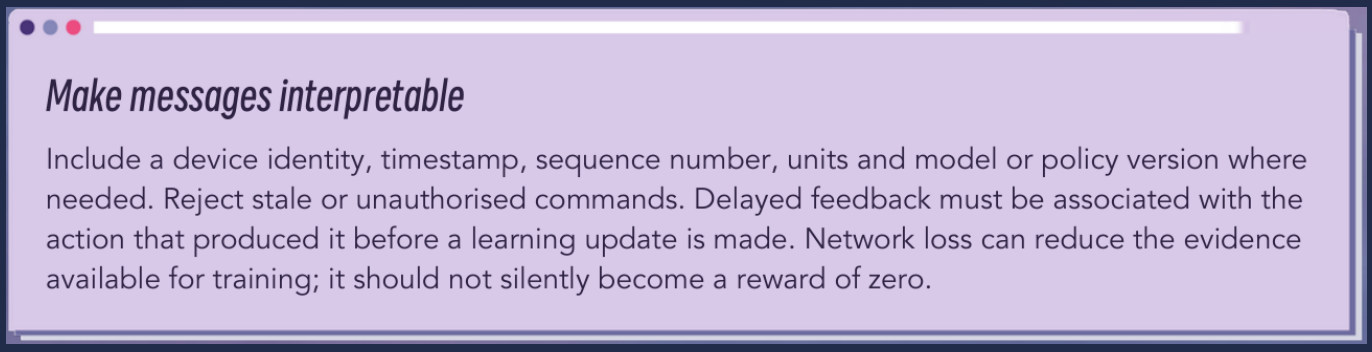
The driver needs immediate awareness; the dispatcher benefits from a broader view. The analogy separates responsibilities.

The network may distribute reviewed policy updates, but a received update should not automatically bypass validation.

Edge and cloud are not opposing teams. Put each part of the workload where its deadlines and resource needs fit.

A number needs context

Video 03:44–04:33 / Study guide page 75



Exact excerpt from the Blue Impulse study guide

Look at the final paragraph on page seventy-five. A message should carry enough context to be interpreted correctly.

Device identity tells us which source or destination it concerns. A timestamp helps determine freshness.

A sequence number can expose repeated or out-of-order messages. Units prevent confusing metres with centimetres.

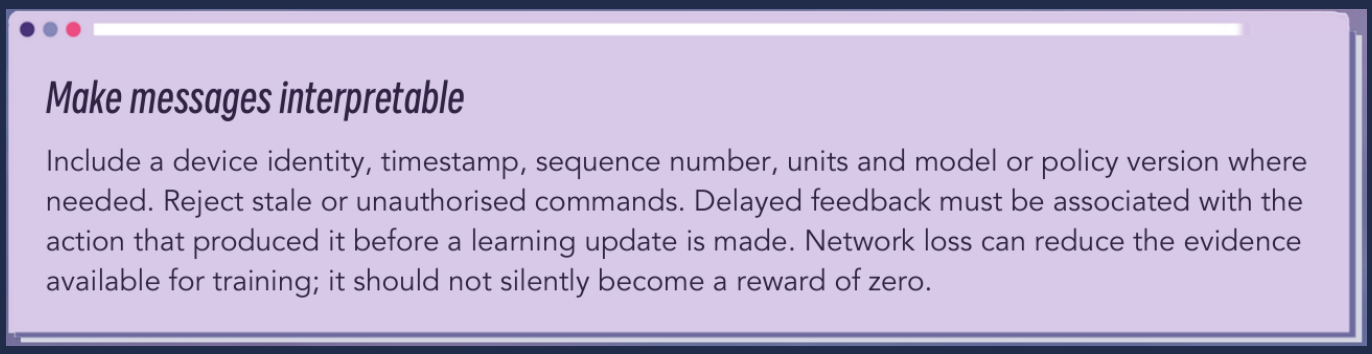
A policy version connects an action or update to the model that produced it.

These fields are useful evidence, but fields alone do not authenticate a sender. Security checks must support the claimed identity.

Before acting, the receiver needs a policy for rejecting stale, unexpected, or unauthorised commands.

Inspect three incoming messages

Video 04:33–05:42 / Study guide page 75



Exact excerpt from the Blue Impulse study guide

Let's run the supporting packet-checking example. This is an illustrative design, not a universal protocol from the book.

Our toy receiver expects robot seven, metres as the unit, and policy version three.

For this demonstration, it rejects commands older than two hundred milliseconds and sequences already processed.

The first message has the expected context, a new sequence, and an age of fifty milliseconds. It passes these checks.

The second is six hundred milliseconds old. The receiver rejects it as stale, even if the network eventually delivered it.

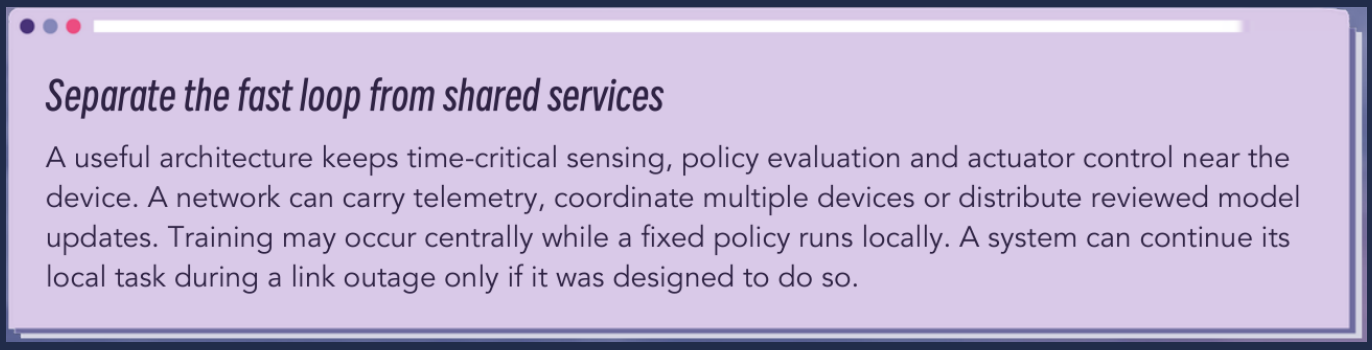
The third repeats an already processed sequence. The receiver rejects it as a duplicate instead of executing the command twice.

Our example assumes authentication was checked elsewhere. It does not implement cryptographic authentication or certify a safety-critical controller.

The useful lesson is why delivery alone is insufficient: the receiver must decide whether the message is still meaningful.

Match feedback to the original action

Video 05:42–06:32 / Study guide page 75



Exact excerpt from the Blue Impulse study guide

The guide makes a second point on page seventy-five: delayed feedback must be associated with the action that produced it.

Suppose action forty-two turns left, and action forty-three moves forward. Their feedback packets arrive in the opposite order.

Updating whichever action happened most recently would connect evidence to the wrong decision.

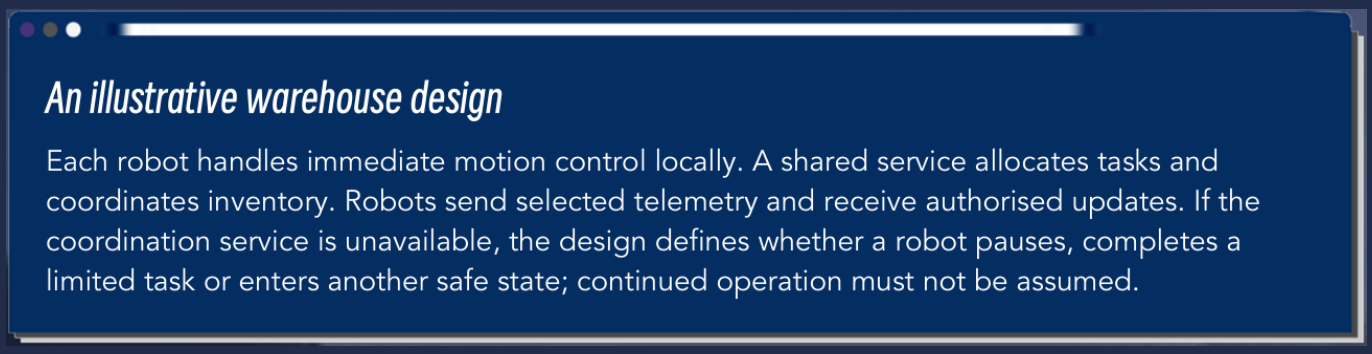
Use identifiers and recorded context to match each result with its original state and action before a learning update.

If a feedback packet is lost, the evidence is missing. That does not silently mean the reward was zero.

This is how networking details can affect the data used by a learning algorithm, even when policy evaluation stays local.

Define what an outage means

Video 06:32–07:20 / Study guide page 89



Exact excerpt from the Blue Impulse study guide

Return to the warehouse paragraph on page eighty-nine and read the outage condition carefully.

The design must say whether the robot pauses, completes a limited task, or enters another defined state.

Continuing safely during a link outage cannot simply be assumed because some computation is local.

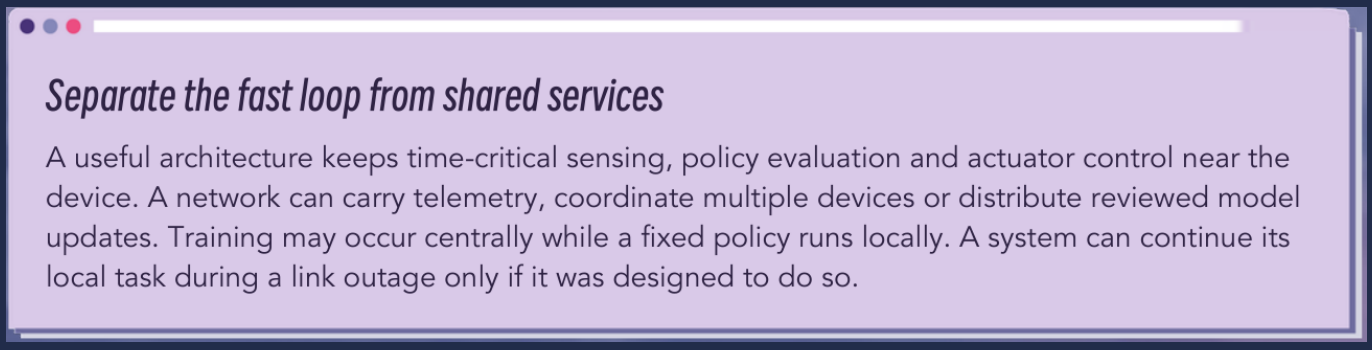
Consider expiring permissions, missing coordination, conflicting routes, and how the system reconnects after the interruption.

Record what happened locally so the shared service can reconcile state when communication returns.

The right fallback depends on the task and its hazards. Our tutorial demonstrates the reasoning rather than certifying a physical robot.

The feedback packet never arrived

Video 07:20–08:12 / Study guide page 75



Exact excerpt from the Blue Impulse study guide

Pause and think. The robot performed action forty-two, but its reward packet never arrived.

Should the learner record a reward of zero? And should a repeated command be executed again automatically?

No to both. Missing evidence is not an observed zero, and a duplicate needs the receiver's defined handling policy.

Associate any later feedback with the original action. Decide how long to wait and how to record missing data.

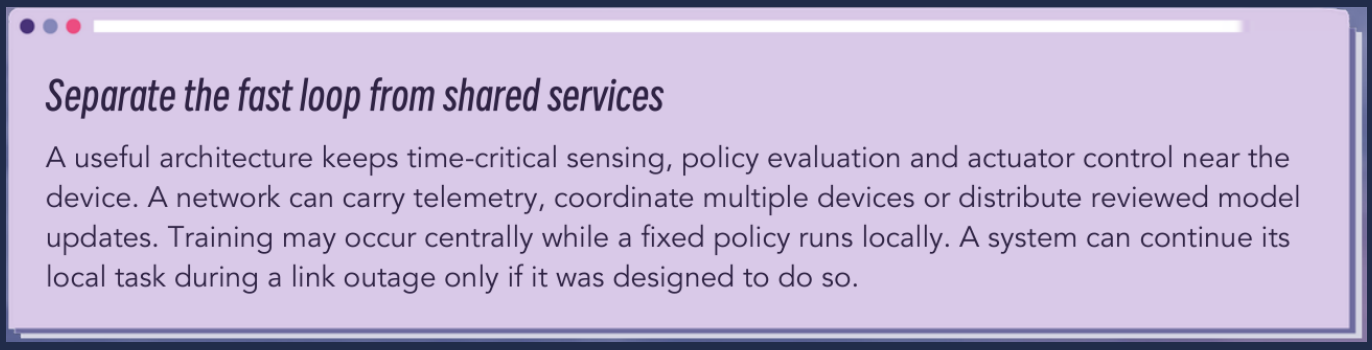
Then explain why a timestamp alone does not prove a message came from an authorised sender.

Use the packet demonstration to change an age or sequence and predict which validation rule should reject it.

Pause first: Missing feedback = reward zero?
Check: No. Missing evidence is not an observed zero.

Connect the responsibilities

Video 08:12–09:07 / Study guide page 75



Exact excerpt from the Blue Impulse study guide

Recap page seventy-five. Keep suitable time-critical work local, and use the network for telemetry, coordination, and reviewed updates.

Messages need interpretable context and appropriate authentication. Stale or repeated commands must not be treated as automatically valid.

Delayed learning feedback belongs to its original action. Lost evidence must not silently become zero reward.

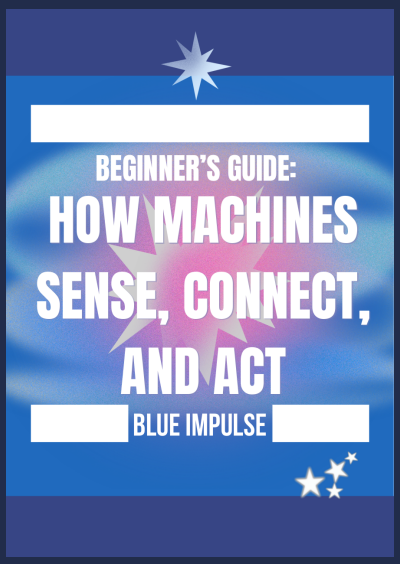
Page eighty-nine adds the deployment question: which part belongs on the robot, at a nearby gateway, or in the cloud?

Finally, distinguish a fixed reactive rule, a deployed learned policy, and an algorithm currently updating that policy.

Draw the local and shared responsibilities, then explain your outage behaviour before calling the architecture complete.

Blue Impulse study guide

Video 09:07–09:38 / Study guide page 1



This tutorial used the Blue Impulse study guide, especially pages twenty-four, twenty-five, seventy-four, seventy-five, and eighty-nine.

The original book cover on screen credits the source of the lesson.

The companion PDF, narration, subtitles, and illustrative packet-checking code are included for your revision.

Blue Impulse study guide. Connect the system, and preserve the meaning of its data.

Program execution

09:32 video · English narration · Guide-linked examples

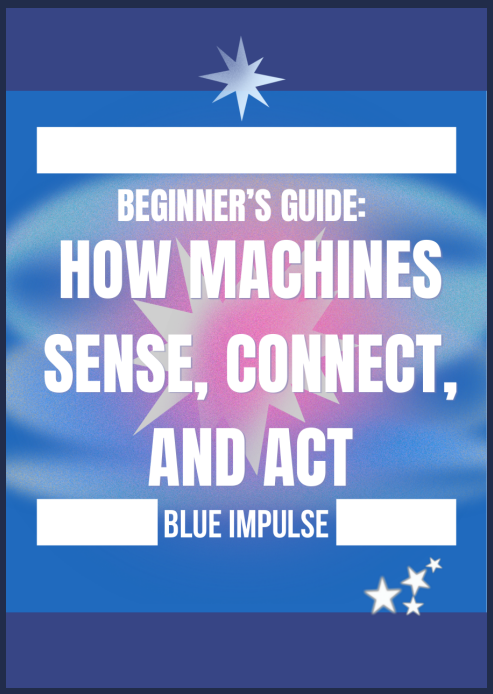
Learn with the book

1. Trace fetch, decode, and execute

2. Give each register its job

3. Follow a four-instruction program

Keep the 96-page Blue Impulse study guide open. Its printed page numbers match the PDF viewer. The screenshots in this tutorial are exact crops from that book.



Guide pages: 12, 13, 14, 15, 39.

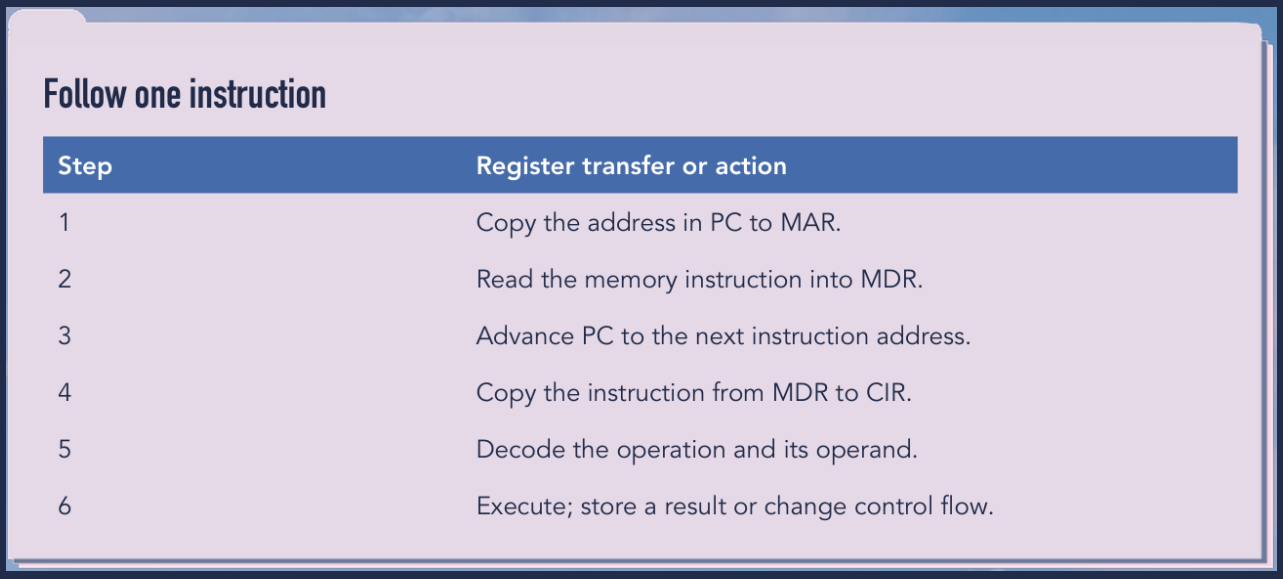
How to use this PDF

Follow the timestamp at the top of each lesson page to find the matching video section. Read the book excerpt, work through the teaching text, and stop at each question before reading the answer. The complete narration is included here, with separate editable scripts and subtitle files in the package.

Practice file: BlueImpulse_Demonstrations.py
Requires Python 3.9+ and no extra packages. The source guide, scripts, subtitle sidecars, and editable video project are included in the tutorial package.

How does a line of code become a result?

Video 00:00–00:35 / Study guide page 13



Follow one instruction

Step	Register transfer or action
1	Copy the address in PC to MAR.
2	Read the memory instruction into MDR.
3	Advance PC to the next instruction address.
4	Copy the instruction from MDR to CIR.
5	Decode the operation and its operand.
6	Execute; store a result or change control flow.

Exact excerpt from the Blue Impulse study guide

Have you ever wondered what changes inside a computer after you press Run?

A program does not execute because the processor understands the whole page like a person reading an essay.

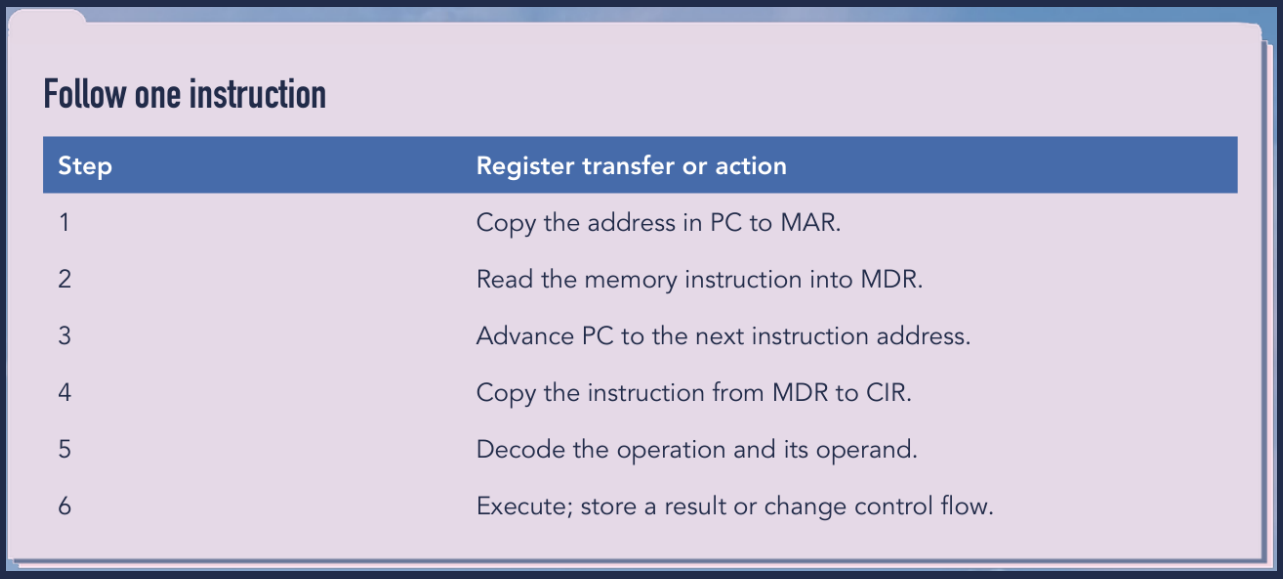
It follows encoded instructions, changing registers, memory, and the next instruction address as it works.

Open the Blue Impulse study guide at page thirteen, Fetch, decode, execute.

We will use that page and the tiny program on page fifteen to track every important state change by hand.

Today's route

Video 00:35–01:13 / Study guide page 13



Follow one instruction

Step	Register transfer or action
1	Copy the address in PC to MAR.
2	Read the memory instruction into MDR.
3	Advance PC to the next instruction address.
4	Copy the instruction from MDR to CIR.
5	Decode the operation and its operand.
6	Execute; store a result or change control flow.

Exact excerpt from the Blue Impulse study guide

You will explain fetch, decode, and execute, and give each teaching-model register a clear job.

You will trace four instructions and distinguish an immediate value from a memory address.

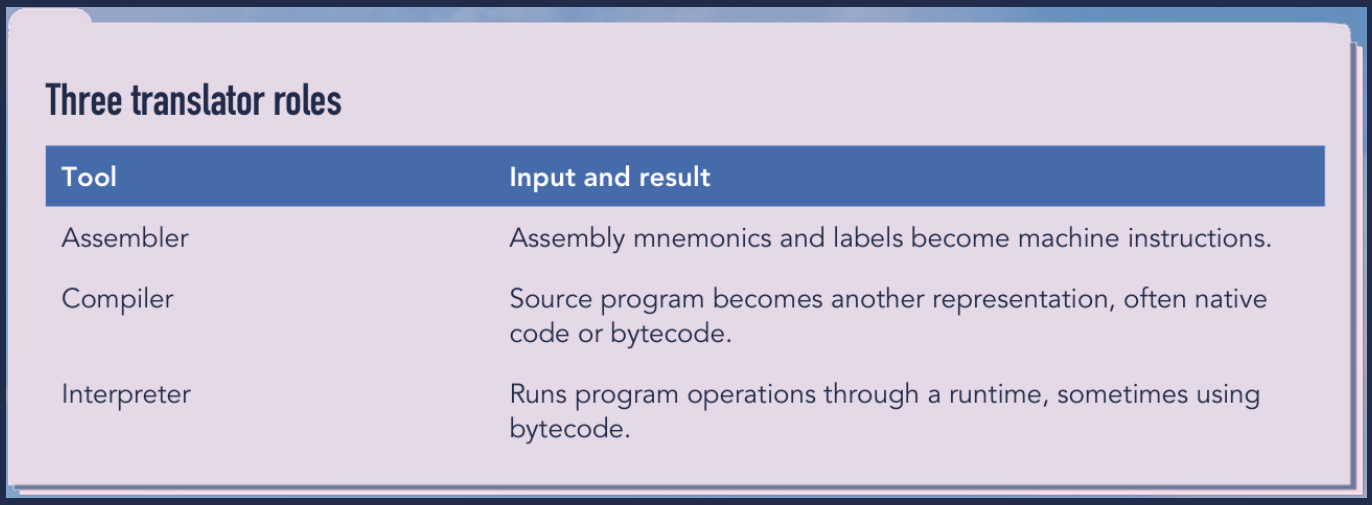
You will also connect source code, translators, and machine instructions without oversimplifying how modern language implementations work.

Welcome to the tutorial on program execution by Blue Impulse.

Have a small table ready with columns for the program counter, current instruction, accumulator, and memory location two hundred.

Before the processor executes

Video 01:13–02:01 / Study guide page 14



Three translator roles

Tool	Input and result
Assembler	Assembly mnemonics and labels become machine instructions.
Compiler	Source program becomes another representation, often native code or bytecode.
Interpreter	Runs program operations through a runtime, sometimes using bytecode.

Exact excerpt from the Blue Impulse study guide

Start with page fourteen, From source code to execution. Machine code contains the instructions the processor executes.

An assembler translates assembly mnemonics and labels. A compiler produces another representation, often native code or bytecode.

An interpreter runs operations through a runtime, sometimes working with bytecode.

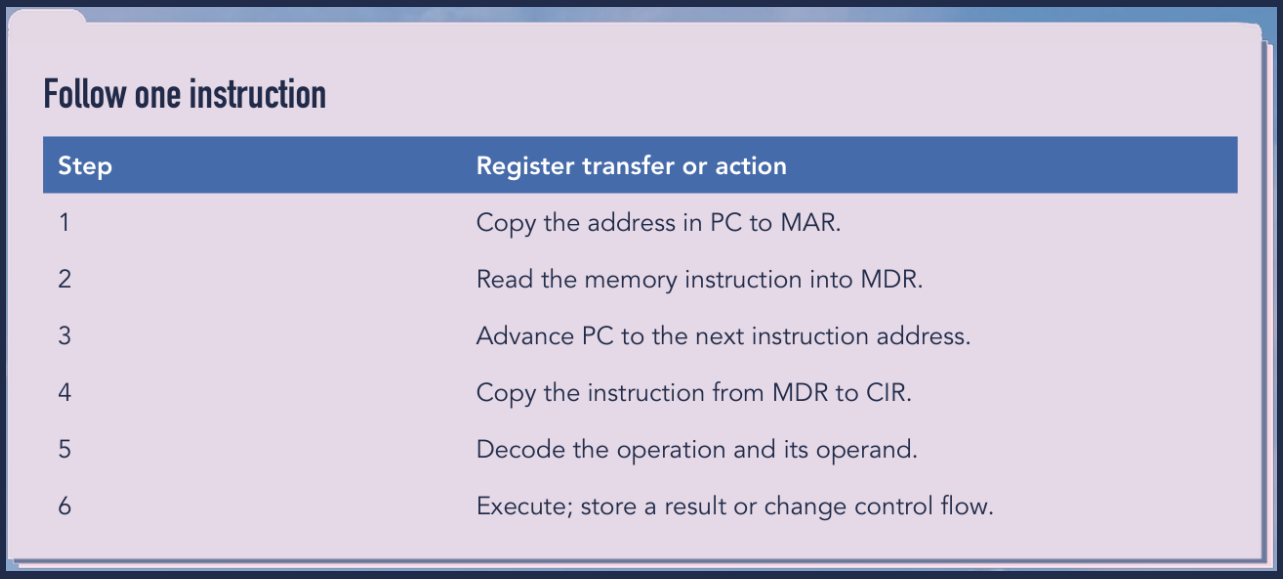
The same language can have compiled and interpreted implementations, and modern systems may combine several techniques.

Avoid the blanket claim that every interpreter translates exactly one source line at a time.

For our demonstration, the book supplies an invented assembly notation so that the state changes remain easy to see.

Use the recipe analogy

Video 02:01–02:51 / Study guide page 13



Follow one instruction

Step	Register transfer or action
1	Copy the address in PC to MAR.
2	Read the memory instruction into MDR.
3	Advance PC to the next instruction address.
4	Copy the instruction from MDR to CIR.
5	Decode the operation and its operand.
6	Execute; store a result or change control flow.

Exact excerpt from the Blue Impulse study guide

Page thirteen uses a recipe to explain the register jobs. The program counter marks the next recipe line.

The memory address register selects the location. The memory data register carries the item read from memory.

The current instruction register holds the instruction being followed. The accumulator is a temporary mixing bowl for arithmetic.

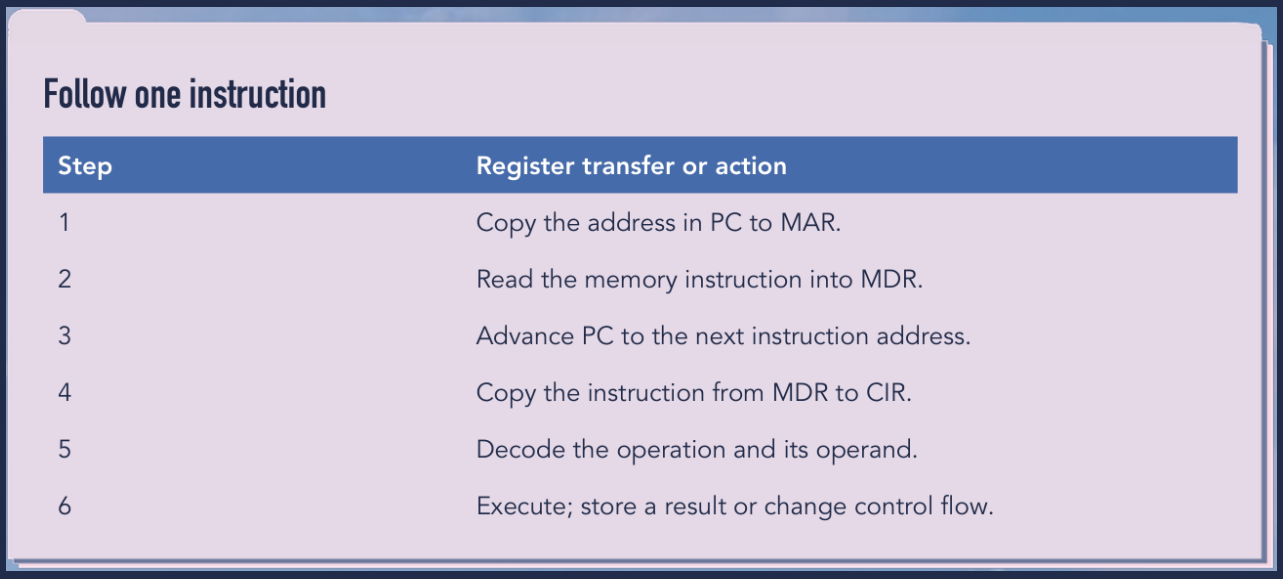
We abbreviate those names as P C, M A R, M D R, C I R, and A C C.

The analogy helps you remember the jobs, but these are electronic storage locations inside a processor model.

Keep the jobs distinct: an address points to a location, while an instruction or value is the content being handled.

Fetch one instruction

Video 02:51–03:43 / Study guide page 13



Follow one instruction

Step	Register transfer or action
1	Copy the address in PC to MAR.
2	Read the memory instruction into MDR.
3	Advance PC to the next instruction address.
4	Copy the instruction from MDR to CIR.
5	Decode the operation and its operand.
6	Execute; store a result or change control flow.

Exact excerpt from the Blue Impulse study guide

Follow the numbered table on page thirteen. First copy the address in the program counter to the memory address register.

Read the instruction at that memory location into the memory data register.

Advance the program counter so it identifies the next instruction address, then copy the fetched instruction into the current instruction register.

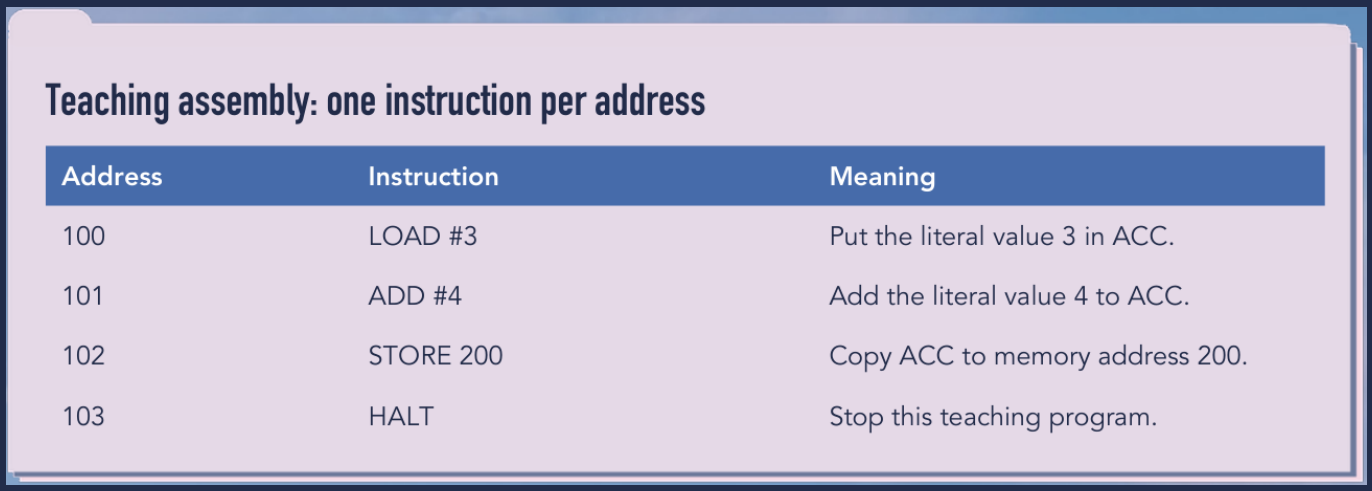
The control unit can now decode the operation and its operand, and the processor executes the required work.

In this simplified model, every instruction occupies one word, so the program counter advances by one.

Real instruction sizes and exact timing depend on the architecture. Our table is a teaching model, not every processor's wiring diagram.

Read the four instructions

Video 03:43–04:27 / Study guide page 15



Teaching assembly: one instruction per address

Address	Instruction	Meaning
100	LOAD #3	Put the literal value 3 in ACC.
101	ADD #4	Add the literal value 4 to ACC.
102	STORE 200	Copy ACC to memory address 200.
103	HALT	Stop this teaching program.

Exact excerpt from the Blue Impulse study guide

Turn to page fifteen. The program starts at address one hundred with LOAD, hash three.

The hash sign denotes an immediate value in this invented notation, so that instruction puts the number three into the accumulator.

Address one hundred and one contains ADD, hash four. Address one hundred and two contains STORE, two hundred.

Address one hundred and three contains HALT, which ends this teaching program.

The number two hundred in STORE identifies a memory location. It is not the value to put into the accumulator.

Before running anything, predict what that memory location will contain when the program finishes.

Single-step the book's program

Video 04:27–05:34 / Study guide page 15

Teaching assembly: one instruction per address

Address	Instruction	Meaning
100	LOAD #3	Put the literal value 3 in ACC.
101	ADD #4	Add the literal value 4 to ACC.
102	STORE 200	Copy ACC to memory address 200.
103	HALT	Stop this teaching program.

Exact excerpt from the Blue Impulse study guide

Let's single-step. Initially, the program counter is one hundred. Fetch the LOAD instruction and advance to one hundred and one.

Decode and execute LOAD, hash three. The accumulator now contains three.

Next fetch ADD, hash four from address one hundred and one. The program counter advances to one hundred and two.

Execution adds four to the accumulator's three, giving seven. Memory location two hundred has not been written yet.

Fetch STORE, two hundred. The program counter advances to one hundred and three, and execution copies seven into memory location two hundred.

The accumulator still contains seven. STORE copies the value; it does not turn the accumulator into an address.

Finally fetch HALT. In this simplified trace, the fetch advances the counter to one hundred and four before execution stops.

Our runnable Python model prints these same states and the register transfers, so you can check the trace against page fifteen.

Value or address?

Video 05:34–06:25 / Study guide page 15

Teaching assembly: one instruction per address

Address	Instruction	Meaning
100	LOAD #3	Put the literal value 3 in ACC.
101	ADD #4	Add the literal value 4 to ACC.
102	STORE 200	Copy ACC to memory address 200.
103	HALT	Stop this teaching program.

Exact excerpt from the Blue Impulse study guide

Pause and check your prediction. After the STORE instruction, what are the accumulator and memory location two hundred?

Also ask yourself whether STORE, two hundred means storing the number two hundred.

The accumulator remains seven, and memory location two hundred receives seven. Two hundred is the destination address.

The hash sign in our LOAD and ADD operands marks literal values. The STORE operand has a different role.

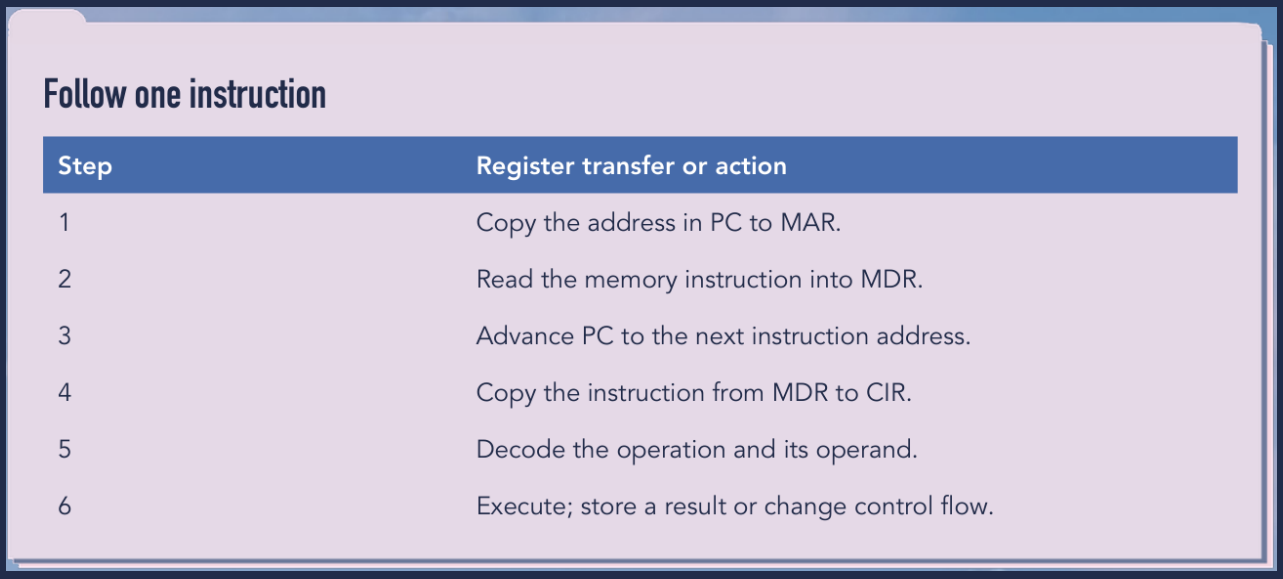
This distinction prevents a common tracing mistake: confusing where data lives with the value stored there.

Use the book's instruction meanings before applying assumptions from a different programming language or processor.

Pause first: After STORE 200: ACC = ? and memory[200] = ?
Check: ACC = 7; memory[200] = 7

The next address can change

Video 06:25–07:14 / Study guide page 13



Follow one instruction

Step	Register transfer or action
1	Copy the address in PC to MAR.
2	Read the memory instruction into MDR.
3	Advance PC to the next instruction address.
4	Copy the instruction from MDR to CIR.
5	Decode the operation and its operand.
6	Execute; store a result or change control flow.

Exact excerpt from the Blue Impulse study guide

Return to page thirteen. It notes that a branch instruction may replace the normal next value of the program counter.

That is how the instruction sequence can follow a decision or repeat a loop instead of always moving forward.

Our four-instruction program contains no branch, so the normal next addresses stay easy to follow.

Page thirty-nine adds pipelining: real processors may overlap work on several instructions while preserving the required program behaviour.

Overlapping the implementation does not make the logical effects of a program arbitrary.

For a basic trace, first understand the defined instruction effects, then study how a specific processor implements them.

Change one instruction

Video 07:14–08:06 / Study guide page 15

Teaching assembly: one instruction per address

Address	Instruction	Meaning
100	LOAD #3	Put the literal value 3 in ACC.
101	ADD #4	Add the literal value 4 to ACC.
102	STORE 200	Copy ACC to memory address 200.
103	HALT	Stop this teaching program.

Exact excerpt from the Blue Impulse study guide

Your turn. Change ADD, hash four to ADD, hash nine, while keeping LOAD, hash three and STORE, two hundred.

What value should reach memory location two hundred? Work through the accumulator before we reveal it.

LOAD gives three. Adding nine gives twelve. STORE copies twelve into memory location two hundred.

That is a small example of testing a prediction by changing one input to the program.

The included simulator accepts a different add value, prints the trace, and checks that the final memory result matches the arithmetic.

Try another value and explain each register's role aloud as the instruction moves through the cycle.

Pause first: LOAD #3; ADD #9; STORE 200
Check: memory[200] = 3 + 9 = 12

Instructions change machine state

Video 08:06–09:01 / Study guide page 13

Follow one instruction

Step	Register transfer or action
1	Copy the address in PC to MAR.
2	Read the memory instruction into MDR.
3	Advance PC to the next instruction address.
4	Copy the instruction from MDR to CIR.
5	Decode the operation and its operand.
6	Execute; store a result or change control flow.

Exact excerpt from the Blue Impulse study guide

Recap the lesson. Source code reaches execution through a translator or runtime appropriate to its implementation.

The program counter identifies the next instruction. Fetch brings it from memory, decode interprets it, and execute performs its operation.

The memory address register selects a location, the memory data register carries data, and the current instruction register holds the operation.

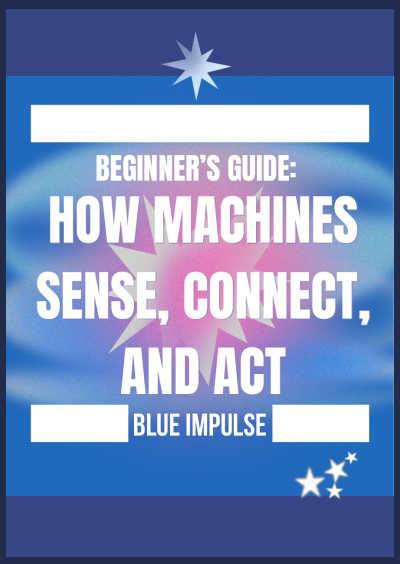
Our teaching accumulator went from three to seven, and STORE copied seven to memory location two hundred.

Addresses and values have different roles. Branches can change the normal next instruction address.

Revisit pages thirteen through fifteen and complete the trace without watching the moving highlight. Understanding means predicting the state changes yourself.

Blue Impulse study guide

Video 09:01–09:32 / Study guide page 1



This tutorial used the Blue Impulse study guide, especially pages twelve through fifteen, with the pipeline connection on page thirty-nine.

The original cover on screen credits the book we followed.

Use the companion tutorial PDF, complete narration, subtitles, and runnable instruction simulator for revision.

Blue Impulse study guide. Follow the instruction, predict the state, and check the result.